



Technical Documentation

Phase 1 & Phase 2 - Complete Reference

Consolidated from /docs - Last verified 2026-08-28

Table of Contents

1. [Project Overview](#)
2. [Architecture](#)
3. [Database](#)
4. [MEGA ID](#)
5. [User Roles](#)
6. [Authentication & Authorization](#)
7. [Product Rules & Architectural Decisions](#)
8. [Academic Sessions](#)
9. [Grades & Promotion](#)
10. [Certificates](#)
11. [Courses & Enrollments](#)
12. [API Reference](#)
13. [Testing](#)
14. [Deployment](#)
15. [Known Gaps & Issues](#)
16. [Development Guidelines](#)
17. [Changelog](#)

MEGA.EDU - Project Overview

Status legend used throughout /docs: **[Implemented]** Implemented - **[Planned]** Designed/approved, not yet implemented - **[Gap]** Known gap/issue - **[Future]** Future/planned Last verified: 2026-08-28, against the current codebase.

What MEGA.EDU is

MEGA.EDU is a national education network connecting schools, teachers, students, parents, and education organizations under one identity system called **MEGA ID**. In plain terms: a school gets a public profile and a digital identity; teachers and students at that school get accounts a school administrator approves; organizations can publish online courses and post opportunities (scholarships, competitions, jobs); parents can follow their children's progress. Everyone - regardless of role - signs in with the same single account. See [MEGA_ID.md](#) and [USER_ROLES.md](#) for the identity/role model in detail.

Overall purpose

The platform exists to give every school, teacher, and student in the network a verifiable digital presence: a school's identity isn't tied to a single administrator's login, a student's learning record and certificates follow *them* rather than staying locked inside one school's paperwork, and achievements (course completions, skills, certificates) are independently verifiable by anyone with the right link - an employer, another school, a scholarship committee.

Major modules currently implemented **[Implemented]**

- **MEGA ID** - one account, one or more roles, held by a real person.
- **Schools & Organizations** - verified directory listings, admin-managed, with staff/member approval workflows.
- **MEGA Academy** - free online courses published by organizations, with modules, lessons, enrollment, and completion tracking.
- **Certificates** - dynamically generated, verifiable credentials issued automatically on course completion, with a public no-login verification page and a designed owner-facing preview.
- **Academic Sessions & Grades ("Phase 2")** - a structured, fully-audited school-year/grade/promotion system: Initial Setup, per-grade Promotion (Promote/Repeat/Transfer/Leave), and New Session rollover with a Pending/Unresolved safeguard so no student's outcome is ever silently guessed or lost.
- **Platform Admin command center** - real, live counts and verification queues for the whole network.
- **Interests & Skills** - self-declared interests, teacher-attested skills.
- **Notifications, Opportunities, Resources, Events, News** - lightweight supporting content types.

Major modules not yet implemented **[Planned]** / **[Future]**

- **Payments** - modeled in the database (Subscription, Payment) but no payment processor is connected; paid course enrollment is explicitly blocked rather than attempted.

- **Certificate PDF export & QR codes** - the certificate's visual design is finished and approved; downloading it as a PDF and generating a scannable verification QR code are both still to build.
- **Grade-based certificates** - the certificate system was built to eventually support them (a reserved field exists), but the actual issuance path doesn't exist yet.
- **Automated tests** - none exist yet; see [TESTING.md](#) for what verification practice is used instead.

Technology stack

- **Framework:** Next.js 14 (App Router), React 18, TypeScript.
- **Database:** Prisma ORM 5.20, SQLite in development, PostgreSQL intended for production (not yet configured - see [DEPLOYMENT.md](#)).
- **Auth:** NextAuth 4, email/password (Credentials provider), JWT sessions.
- **Styling:** Tailwind CSS.
- **Validation:** zod on registration routes.
- **No test framework is installed.**

Current development status

Two phases of work are complete and independently verified against the running application and real database state (not just typechecked):

Phase	Scope	Status
Phase 1	MEGA ID, roles, schools/orgs, MEGA Academy, certificates, Platform Admin dashboard	[Implemented] Complete
Phase 2	Academic Sessions & Grades - schema, matching utility, Initial Setup, Promotion, New Session rollover	[Implemented] Complete, all six steps independently verified with real evidence (timing tests, database-level audit-trail checks, multi-session-chain tests)

Nothing beyond these two phases has been started. No production deployment has occurred.

High-level system architecture

A single Next.js application - pages and API routes live side by side, both talking to one Prisma-managed database. No separate backend service, no microservices.

```
[Diagram source - Mermaid syntax, view in the Markdown files under /docs for a rendered diagram]
graph TD
    subgraph Client ["Browser"]
        UI["Pages & Components<br/>(Server + Client Components)"]
    end
    subgraph App ["Next.js App (single deployment)"]
        Pages["App Router<br/>src/app/**"]
        API["API Routes<br/>src/app/api/**"]
        Lib["src/lib - auth, authorize,<br/>certificates, gradeHistory,<br/>gradeMatching, gradeRollover, notify"]
    end
    Client --- App
```

```
DB["Prisma + SQLite (dev)<br/>/ PostgreSQL (prod, planned)"] UI -->|"navigation, form  
POSTs"| Pages UI -->|"fetch()"| API Pages --> Lib API --> Lib Lib --> DB
```

See [ARCHITECTURE.md](#) for the detailed breakdown of this diagram, including auth and module relationships.

Where to go next

- Building or reviewing a feature? Start with [ARCHITECTURE.md](#) for the conventions this codebase actually follows.
- Need the exact shape of the database? [DATABASE.md](#).
- Wondering whether a decision was deliberate? [PRODUCT_RULES.md](#) documents every explicitly-approved rule with its rationale.
- Doing AI-assisted development on this codebase? Read [DEVELOPMENT_GUIDELINES.md](#) first.
- Wondering what's broken or missing? [KNOWN_GAPS.md](#).

Architecture

Status legend: **[Implemented]** Implemented - **[Planned]** Designed/approved, not yet implemented - **[Gap]** Known gap/issue - **[Future]** Future/planned Last verified: 2026-08-28, against the current codebase.

High-level shape **[Implemented]**

MEGA.EDU is a single Next.js 14 App Router application - no separate backend service. Pages and API routes live side by side under `src/app`, both talking to the same Prisma client (`src/lib/prisma.ts`) against one SQLite database file (`prisma/dev.db` in dev).

```
src/ app/ # pages (App Router) + API routes, colocated api/ # route.ts handlers, grouped by
resource dashboard/ # role-specific dashboard pages/components setup/ # Phase 2: Initial
School Setup wizard grades/ # Phase 2: per-grade Promotion rosters + Pending queue
sessions/new/ # Phase 2: New Session rollover admin/ # Platform Admin-only pages ... # public
pages: schools, courses, opportunities, etc. components/ # shared client components
(SiteHeader, DashboardHero, certificate/) lib/ # server-side helpers: auth, authorize,
prisma, notify, # certificates, certificateView, gradeHistory, # gradeMatching, gradeRollover
types/ # ambient type augmentation (next-auth.d.ts) prisma/ schema.prisma # single source of
truth for the data model seed.ts # idempotent demo data + platform fixtures (upserts)
```

System diagram

```
[Diagram source - Mermaid syntax, view in the Markdown files under /docs for a rendered
diagram] flowchart TB
  subgraph Browser
    SC["Server Components<br/>(most pages)"]
    CC["Client Components<br/>(dashboards, wizards, forms)"]
  end
  subgraph NextApp["Next.js App"]
    Pages["Pages<br/>src/app/**/page.tsx"]
    API["API Routes<br/>src/app/api/**/route.ts"]
  end
  subgraph Lib["src/lib"]
    Auth["auth.ts<br/>NextAuth config"]
    Authorize["authorize.ts<br/>requireSchoolAdmin, requireOrgAdmin,<br/>requirePlatformAdmin,
    requireSchoolFinance,<br/>requireOrgFinance, requireCourseOwner"]
    Cert["certificates.ts /
    certificateView.ts<br/>issueCourseCertificate()"]
    Grade["gradeHistory.ts / gradeMatching.ts /
    gradeRollover.ts<br/>recordGradeDecision(),
    matchLegacyGradeText(),<br/>carryForwardEligibleStudents()"]
    Notify["notify.ts"]
  end
  Prisma["Prisma Client"]
  DB["SQLite (dev)<br/>PostgreSQL (prod, planned)"]
  SC -->|"prisma.* queries directly"| Pages
  CC -->|"fetch()"| API
  Pages --> Lib
  API --> Lib
  Lib --> Prisma
  Prisma --> DB
```

Rendering pattern **[Implemented]**

Two consistent patterns are used throughout, not mixed within one page:

1. **Server component fetches Prisma directly, renders (or passes props to) a client component.** The dominant pattern for anything showing a logged-in user their own data: `dashboard/page.tsx` branches on role, runs the relevant Prisma query, and hands a fully-hydrated object to a `"use client"` component. All of Phase 2's pages (`setup/page.tsx`, `grades/page.tsx`, `grades/[schoolGradeId]/page.tsx`, `sessions/new/page.tsx`) follow this exact pattern - resolve the admin's school server-side, fetch everything the page needs, pass typed props to a client component that owns the interactive state.

2. **Client component fetches its own API route.** Used for mutations and independently-pollled data (`StudentSkillManager`, the notification bell). The mutation flow is consistently: `fetch()` -> check `res.ok` -> `router.refresh()` (server components re-run, client state stays put).

There is no client-side global state manager - `router.refresh()` is the app's entire "revalidate after mutation" strategy.

The `src/lib` layer - "the only path" pattern [Implemented]

A recurring, deliberate decision (see [PRODUCT_RULES.md](#)): for any write that must be atomic with a side effect or must never be bypassed, the codebase centralizes it into one function, and every caller goes through it rather than calling `prisma.<model>.update` directly:

- `issueCourseCertificate()` (`certificates.ts`) - the only place a `Certificate` row is created for a course.
- `recordGradeDecision()` (`gradeHistory.ts`) - the only place a `GradeHistory` row's status/outcomeGradeId may change, always paired with a `GradeHistoryAudit` insert.
- `carryForwardEligibleStudents()` (`gradeRollover.ts`) - the only place new-session placements get auto-created from a prior decision; idempotent and re-runnable.

All three follow the same shape: typed input, optional `tx?: Prisma.TransactionClient` for composing into a larger transaction, transactional by default otherwise.

Other `lib` modules: `auth.ts` (NextAuth config), `authorize.ts` (the `requireX` guard suite - see [AUTHENTICATION_AND_AUTHORIZATION.md](#)), `prisma.ts` (singleton client), `notify.ts` (best-effort notifications, never allowed to fail the calling action), `certificateView.ts` (pure view-model builder, no live text lookups), `gradeMatching.ts` (`matchLegacyGradeText()`) - never guesses, returns `null`.

Major application modules and how they relate [Implemented]

```
[Diagram source - Mermaid syntax, view in the Markdown files under /docs for a rendered diagram]
flowchart LR
    MEGAID["MEGA ID<br/>(User, UserRole)"]
    School["School module"]
    Org["Organization module"]
    Academy["MEGA Academy<br/>(Course, Enrollment)"]
    Cert["Certificates"]
    Grades["Academic Sessions & Grades<br/>(Phase 2)"]
    MEGAID -->|"one person, many roles"| School
    MEGAID -->|"one person, many roles"| Org
    MEGAID -->|"enrolls as Teacher/Student"| Academy
    Academy -->|"completion issues"| Cert
    School -->|"scopes"| Grades
    Grades -->|"Student.userId anchors"| Org
    Org -->|"publishes, may issue"| Cert
    School -->|"may issue / associate"| Cert
```

A school and an organization are structurally independent - a `Course` belongs to an `Organization`, never a `School`, and Phase 2's grade structure belongs entirely to a `School`, with no organization involvement. The two only meet at `Certificate.associatedSchoolId`, an informational link (the recipient's school), not an ownership relation.

Data model conventions [Implemented]

See [DATABASE.md](#) for the full model inventory. Two conventions apply project-wide:

- **No Prisma enums, ever** - SQLite's connector doesn't support them, even unused ones.

- **Snapshot fields for anything that must survive a later rename** - `Certificate.*NameSnapshot` and `GradeHistoryAudit.previous*/new*` are plain values, not live FK relations. Logos are the one deliberate exception (looked up live). Full rationale in [PRODUCT_RULES.md](#).

Migration discipline [Implemented]

Both Phase 1 and Phase 2 followed the same additive-first sequence, enforced manually (not by tooling):

1. Add new models/fields additively - nothing existing removed in the same pass.
2. Before applying anything that could conflict with existing data, write a one-off verification script, run it, confirm no conflicts, delete the script.
3. Apply with `npx prisma db push` (no `prisma migrate/migrations/` folder - appropriate for the current single-environment SQLite setup).
4. Only after a change is applied and verified does any "clean up the old thing" pass happen - and some legacy fields (`Student.gradeLevel`) are intentionally never dropped.

Testing [Future]

No test framework is installed. See [TESTING.md](#) for what verification practice is used instead.

Deployment [Future]

See [DEPLOYMENT.md](#) - no production infrastructure exists yet; only local development is configured.

Database

Status legend: **[Implemented]** Implemented - **[Planned]** Designed/approved, not yet implemented - **[Gap]** Known gap/issue - **[Future]** Future/planned Last verified: 2026-08-28, against `prisma/schema.prisma` directly.

Datasource: SQLite in development (`prisma/dev.db`); `.env.example` and the schema's own header comment both mark PostgreSQL as the intended production target (nothing production-specific is configured yet - see [DEPLOYMENT.md](#)). **No Prisma enums are used anywhere** - SQLite's connector doesn't support them, even unused ones - every status/type/role field is a plain `String`, with valid values documented in a comment above the field.

Every model below is **[Implemented]** implemented (exists, migrated, and has at least one route reading/writing it) unless marked otherwise.

MEGA ID **[Implemented]**

User

Purpose: the single identity record - one MEGA ID, one login, potentially many roles. **Currently used:** yes, everywhere; every other model traces back to a User for who did what. **Key fields:** `id`, `email` (unique), `passwordHash`, `name`, `createdAt`, `updatedAt`. **Relationships:** roles (`UserRole[]`), profile records (`teacherProfile`, `studentProfile`, `parentProfile` - each optional 1:1), admin/accountant links, `subscriptions`, `payments`, `notifications`, `interests`, `skillsAdded`, `instructorProfile`, `certificatesReceived`, `gradeDecisionsMade`, `gradeHistoryAuditsMade`. **Constraints:** email unique. **Delete behavior:** deleting a User cascades to `UserRole`, `Teacher`, `Student`, `Parent`, admin/accountant links, `Interest`, `Notification` (all `onDelete: Cascade`) - but there is no delete route anywhere in the app today; this is schema-level behavior only, never exercised by any UI action.

UserRole

Purpose: which role(s) a User holds. **Currently used:** yes, read on every login and every authorization check. **Key fields:** `id`, `userId` (FK), `role` (String). Valid values: `PLATFORM_ADMIN` | `SCHOOL_ADMIN` | `TEACHER` | `STUDENT` | `PARENT` | `ORGANIZATION_ADMIN` | `ACCOUNTANT`. **Constraints:** `@@unique([userId, role])` - a user can't hold the same role twice, but can hold several different roles. **Delete behavior:** cascades from User.

School **[Implemented]**

School

Purpose: a verified, listed education institution. **Currently used:** yes, extensively. **Key fields:** `id`, `name`, `slug` (unique), `type?`, `location?`, `district?`, `contactEmail?`, `contactPhone?`, `gradesOffered?`, `description?`, `logoUrl?`, `coverImageUrl?`, `verified` (default false), `isActive` (default true), `subscriptionTier` (default "FREE"). **Relationships:** admins, accountants, teachers, students, programs, news, opportunities, resources, subscriptions, issued/associated certificates, `academicSessions`, grades

(SchoolGrade[]). **Constraints:** slug unique. **Delete behavior:** no delete route exists for School at all. **Notes:** verified is set only by a Platform Admin. [Gap] isActive is read in two places (schools/search filter, Platform Admin dashboard counts) but **nothing in the app ever sets it to false** - there is no deactivation action. logoUrl exists but is unpopulated on every school in the current database and has no upload UI. gradesOffered is unstructured free text, unrelated to the Phase 2 SchoolGrade model.

SchoolAdmin / SchoolAccountant

Purpose: join tables granting School Admin / Accountant access to a specific school. **Currently used:** yes. **Key fields:** userId, schoolId. **Constraints:** @@unique([userId, schoolId]) each - a school can have multiple admins/accountants. **Delete behavior:** cascades from both User and School. **Notes:** SchoolAccountant is granted directly by a School Admin (POST /api/schools/[id]/accountants) - there's no self-registration or approval queue for this role.

Program, NewsPost

Simple school-owned content, no approval workflow. Currently used (School Admin dashboard).

People [Implemented]

Teacher

Purpose: a teacher's profile at (optionally) one school. **Currently used:** yes. **Key fields:** id, userId (unique FK), schoolId?, bio?, subjects?, position (default "Teacher"), approved (default false). **Relationships:** courseEnrollments, gradeAssignments (TeacherGradeAssignment[]). **Delete behavior:** cascades from User. **Notes:** unaffiliated until joining a school; re-joining always resets approved to false.

Student

Purpose: a student's profile at (optionally) one school - also the anchor for Phase 2 grade placement. **Currently used:** yes, heavily. **Key fields:** id, userId (unique FK), schoolId?, gradeLevel?, approved (default false). **Relationships:** parents (ParentStudent[]), courseEnrollments, skills, gradeHistory (GradeHistory[]). **Delete behavior:** cascades from User. **Notes:** gradeLevel is the **legacy free-text grade** - permanently retained as a fallback, no longer written to once a school completes Initial Setup, never scheduled for removal (see [PRODUCT_RULES.md](#)).

Parent, ParentStudent

Purpose: a parent's linked children. **Currently used:** yes. **Constraints:** @@unique([parentId, studentId]). Linking requires the child to already exist as a Student (matched by email). **Delete behavior:** cascades from both sides.

Academic Sessions & Grades - Phase 2 [Implemented] (fully implemented and in active use)

All six models below are pushed to the database **and** actively read/written by real routes - this is no longer schema-only. See [ACADEMIC_SESSIONS.md](#) and [GRADES_AND_PROMOTION.md](#) for the full behavioral write-up; this section covers structure only.

[AcademicSession](#)

Purpose: one school-year window for a school. **Currently used:** yes - Initial Setup, Promotion, and New Session rollover all depend on it. **Key fields:** id, schoolId (FK), name, startDate, endDate, status (default "ACTIVE"). Valid status: ACTIVE | CLOSED. **Constraints:** one ACTIVE session per school, enforced at the application level (not a DB constraint - SQLite can't express a partial unique index here). **Delete behavior:** cascades from School; no delete route exists (sessions are closed, never deleted).

[GradeReference](#)

Purpose: the platform-wide, fixed grade ladder. **Currently used:** yes - seeded once (PP1-PP3, Y1-Y10, 13 rows), read by every grade-configuration screen. **Key fields:** id, code (unique), order (unique Int). **Delete behavior:** no delete route; not school-editable at all.

[SchoolGrade](#)

Purpose: a school's opt-in to one GradeReference, with its own display label. **Currently used:** yes. **Key fields:** id, schoolId (FK), gradeReferenceId (FK), displayName. **Constraints:** @@unique([schoolId, gradeReferenceId]). **Delete behavior:** cascades from School; the creation route (POST /api/schools/[id]/grades) is additive-only - it never deletes a SchoolGrade a school previously opted into.

[TeacherGradeAssignment](#)

Purpose: per-session teacher-to-grade link. **Currently used:** yes. **Key fields:** id, teacherId (FK), schoolGradeId (FK), academicSessionId (FK). **Constraints:** @@unique([teacherId, schoolGradeId, academicSessionId]). **Delete behavior:** cascades from Teacher; a DELETE route exists (/api/schools/[id]/teacher-assignments/[assignmentId]) for removing a single mistaken assignment. Never auto-carried-forward to a new session.

[GradeHistory](#)

Purpose: a student's grade placement for one session - the permanent record. **Currently used:** yes, the central Phase 2 table. **Key fields:** id, studentId (FK), schoolGradeId (FK), academicSessionId (FK), status (default "ENROLLED"), enrolledAt, decidedAt?, decidedByUserId? (FK), outcomeGradeId? (FK to SchoolGrade). Valid status: ENROLLED | COMPLETED | REPEATED | TRANSFERRED | LEFT. **Constraints:** @@unique([studentId, academicSessionId]) - one placement per student per session. **Delete behavior:** cascades from Student; **no delete route exists anywhere** - rows are permanent by design, a repeat or promotion creates a new row in the next session rather than editing this one. **Critical rule:** status/outcomeGradeId may only ever be written through recordGradeDecision() (src/lib/gradeHistory.ts) - see [PRODUCT_RULES.md](#).

[GradeHistoryAudit](#)

Purpose: append-only log of every decision made on a GradeHistory row. **Currently used:** yes - one row per decision, verified 1:1 in multiple live tests. **Key fields:** id, gradeHistoryId (FK), changedByUserId (FK), changedAt, previousStatus, previousOutcomeGradeId?, newStatus, newOutcomeGradeId?. **Constraints:** none beyond FKs. previousOutcomeGradeId/newOutcomeGradeId are deliberately plain nullable strings, **not** live FK relations to SchoolGrade - a frozen snapshot, for the same reason Certificate's *NameSnapshot fields are

plain strings. **Delete behavior:** cascades from `GradeHistory`; no update or delete route ever touches this table - `recordGradeDecision()` is the sole writer and only ever inserts.

Organizations

`Organization` [Implemented]

Purpose: a verified, listed education organization that can publish MEGA Academy courses. **Currently used:** yes. **Key fields:** `id`, `name`, `slug` (unique), `description?`, `website?`, `verified` (default false), `isActive` (default true). **Delete behavior:** no delete route exists. **Notes:** [Gap] **no `logoUrl` field exists at all** - unlike `School`, an organization has no way to have a logo in the schema today. `isActive` has the same "read but never set" characteristic as `School.isActive`.

`OrganizationAdmin` / `OrganizationAccountant` [Implemented]

Same shape and behavior as `SchoolAdmin`/`SchoolAccountant`. `@@unique([userId, organizationId])` each.

Educational Approaches [Implemented]

`EducationalApproach` (e.g. "STEM", "Montessori") and `SchoolApproach` (join table, `@@unique([schoolId, approachId])`) - simple tagging, cross-cutting schools, courses, and resources. Currently used.

MEGA Academy - courses, enrollment, certificates [Implemented]

`Course`

Purpose: a MEGA Academy course. **Currently used:** yes. **Key fields:** `id`, `organizationId?`, `approachId?`, `instructorId?`, `title`, `slug` (unique), `description?`, `priceCents` (default 0), `published` (default false). **Delete behavior:** no delete route exists. **Notes:** `organizationId` is technically optional in the schema, but every real creation path requires an org admin. No "course bundle" or "premium tier" concept exists anywhere - see [PRODUCT_RULES.md](#).

`CourseModule`, `Lesson` [Implemented]

Ordered content under a course. Cascades from `Course`/`CourseModule` respectively.

`CourseEnrollment` [Implemented]

Purpose: one learner's enrollment in one course. **Currently used:** yes. **Key fields:** `id`, `courseId` (FK), `teacherId?`, `studentId?`, `progress` (default 0), `completedAt?`, `enrolledAt`. Either `teacherId` or `studentId` is set, never neither. **Delete behavior:** cascades from `Course`. Has an optional 1:1 `certificate`.

`Certificate` [Implemented]

Purpose: a dynamically-generated, verifiable credential. **Currently used:** yes, issued automatically on course completion. **Key fields:** `id`, `verificationCode` (unique), `recipientUserId` (FK), `enrollmentId?` (unique FK), `gradeHistoryId?` (unique, reserved for Phase 2 - no live Prisma relation declared), `instructorId?` (FK), `issuerType`, `issuerOrganizationId?` (FK), `issuerSchoolId?` (FK), `associatedSchoolId?` (FK), `title`, `recipientNameSnapshot`, `recipientMegaIdSnapshot`, `issuerNameSnapshot`, `associatedSchoolNameSnapshot?`, `instructorNameSnapshot?`, `issuedAt`. Valid `issuerType`: MEGA_EDU | ORGANIZATION | SCHOOL | JOINT (only ORGANIZATION is reachable today). **Delete behavior:** no delete route - certificates are permanent records. **Notes:** full design rationale in [CERTIFICATES.md](#).

Instructor [Implemented]

Purpose: a course/certificate instructor, deliberately decoupled from User. **Currently used:** yes. **Key fields:** `id`, `name`, `megaIdUserId?` (unique FK). Can be named with no MEGA ID at all ("Shown on certificates issued for this course. If they later get a MEGA ID, their record can be linked to it.").

Identity layer - Interests & Skills [Implemented]

Interest

Purpose: a user's self-declared interests. **Currently used:** yes. **Constraints:** `@@unique([userId, name])`. Self-managed by any user. **Delete behavior:** cascades from User.

Skill

Purpose: a skill credited to a student by a teacher/school-admin. **Currently used:** yes. **Key fields:** `id`, `studentId` (FK), `addedByUserId` (FK), `name`, `createdAt`. **Constraints:** `@@unique([studentId, addedByUserId, name])` - prevents the *same person* crediting the *same skill* to the *same student* twice, while different people independently crediting the same skill is fully allowed (each is a separate row). See [PRODUCT_RULES.md](#). **Delete behavior:** cascades from Student. The creation route catches the resulting P2002 and returns `{ok: true, alreadyExists: true}` rather than an error.

Interest vs. Skill, in one line: an Interest is self-declared and belongs to any User; a Skill is teacher-attested and belongs specifically to a Student, crediting who added it.

Resources, Events, Opportunities [Implemented]

Resource, Event, Opportunity - optionally attached to a School and/or Organization (both nullable FKs). No approval/moderation workflow - posting is publishing.

Subscriptions & Payments [Planned] (modeled, not integrated)

Subscription (`id`, `userId?`, `schoolId?`, `plan`, `status` (default "ACTIVE"), `startedAt`, `endsAt?`) and Payment (`id`, `userId?`, `subscriptionId?`, `amountCents`, `currency` (default "NPR"), `status` (default "PENDING"), `provider?`, `providerRef?`) are fully modeled in the schema but **no payment processor is integrated**. `AccountantDashboard.tsx` tells Accountant-role users plainly that real payment processing isn't built

yet. Course enrollment for a priced course (`priceCents > 0`) is explicitly blocked with an error rather than attempting a charge.

Notifications [Implemented]

Notification - `id`, `userId` (FK), `type`, `title`, `body?`, `read` (default `false`). Written only through `notify()/notifySchoolCommunity()` (`src/lib/notify.ts`), never directly. Current `type` values in use: `SCHOOL_ANNOUNCEMENT`, `STAFF_APPROVED`, `STUDENT_APPROVED`, `CERTIFICATE_ISSUED`, `SCHOOL_VERIFIED`, `ORGANIZATION_VERIFIED`.

Cross-cutting decisions

- **Snapshot fields for anything that must survive a later rename:** `Certificate.*NameSnapshot`, `GradeHistoryAudit.previous/newOutcomeGradeId`. Logos are the deliberate exception (live-looked-up). Full rationale: [PRODUCT_RULES.md](#).
- **`Student.gradeLevel` is permanent legacy fallback**, not scheduled for removal.
- **No cascading deletes on cross-reference relations** (e.g. `GradeHistory.schoolGradeId`, `Certificate.issuerOrganizationId`) - only genuine ownership chains cascade.
- **No model in this schema has a working delete route in the application today** - every "delete behavior" described above is schema-level cascade behavior that would apply *if* a delete ever happened, not something any current UI action triggers (`TeacherGradeAssignment` is the one exception with a real `DELETE` route).

MEGA ID

Status legend: **[Implemented]** Implemented - **[Planned]** Designed/approved, not yet implemented - **[Gap]** Known gap/issue - **[Future]** Future/planned Last verified: 2026-08-28, against the current codebase.

The core principle: MEGA ID belongs to the individual **[Implemented]**

A MEGA ID is a `User` row - one email, one password, one identity. It is **not owned by a school, an organization, or any institution**. A school doesn't issue a MEGA ID to a student the way it might issue a library card; the person creates their own account, and *then* chooses to associate it with a school, an organization, or MEGA Academy. That association can change, end, or expand over time without the identity itself changing.

The doc comment above `authOptions` in `src/lib/auth.ts` states this plainly:

"MEGA ID - one unified login for every role (School Admin, Teacher, Student, Parent, Organization Admin, Platform Admin). A single User can hold multiple roles at once (see `UserRole` in the Prisma schema); the session exposes all of them so the UI can show the right dashboards/actions without a second login."

How a person relates to a School, an Organization, and MEGA Academy **[Implemented]**

These are three separate, optional relationships a single `User` can hold simultaneously - none of them define the identity, they're attached to it:

- **School** - via a `Teacher` or `Student` profile (`schoolId` on that profile, not on the `User`), or via `SchoolAdmin/SchoolAccountant` join rows. A person's `Teacher/Student` profile can be unaffiliated (`schoolId: null`), affiliated and pending approval, or affiliated and approved - the school relationship is a state on the profile, not baked into the account.
- **Organization** - via `OrganizationAdmin/OrganizationAccountant` join rows, granting management access to a specific organization's courses and content.
- **MEGA Academy** - via `CourseEnrollment` rows, tied to the person's `Teacher` or `Student` profile, independent of which school (if any) that profile is affiliated with. Enrolling in a course has no relationship to school affiliation at all - an unaffiliated student can enroll in a free MEGA Academy course.

Multiple roles on one identity **[Implemented]**

Nothing in the schema or auth layer limits a `User` to one role. `UserRole` is a separate table (`@@unique([userId, role])`, not a single field on `User`), and a person can hold several roles at once - e.g. a `Teacher` at one school who is also a `Parent` of a student at another. See [USER_ROLES.md](#) for what each role actually grants and how the dashboard picks which one to show when a person has more than one.

Registration is deliberately two-stage **[Implemented]**

Every registration path creates a **minimal, single-role account first**; affiliation with a school/organization happens later, from the person's own dashboard. Two ways to register:

1. **Generic** (POST /api/auth/register) - name, email, password, one role. Teacher/Student/Parent get an unaffiliated profile immediately; School Admin/Organization Admin get nothing yet.
2. **Role-specific, single-step** - combines account creation with immediate affiliation (requires an already-verified school for Teacher/Student).

Post-registration affiliation routes: POST /api/teacher/join-school, POST /api/student/join-school, POST /api/parent/link-child, POST /api/schools/create-for-admin, POST /api/organizations/create-for-admin.

Learning history follows the person, not the school [Implemented]

This is the practical consequence of the identity model, and it's structural, not just a policy statement:

- **CourseEnrollment** links to Teacher.id/Student.id, which link to User.id - a completed course and its resulting Certificate stay tied to the person regardless of what happens to their school affiliation afterward.
- **GradeHistory** (Phase 2) links to Student.id -> User.id, the same way. A student's placement, promotion, and audit history exist independent of the school relationship - if a student later transfers or leaves (TRANSFERRED/LEFT in GradeHistory.status), their historical GradeHistory rows from that school are never deleted; the record simply notes they left, permanently.
- **Certificate.recipientUserId** + recipientMegaIdSnapshot - a certificate belongs to the recipient's MEGA ID, frozen at issuance, and is reachable by that person regardless of which organization issued it or what that organization is called today.

Authentication [Implemented]

NextAuth Credentials provider: email + password, bcrypt.compare against User.passwordHash. JWT session strategy (not database sessions). Full detail in [AUTHENTICATION_AND_AUTHORIZATION.md](#).

Where MEGA ID surfaces elsewhere [Implemented]

- **Certificates** - Certificate.recipientMegaIdSnapshot freezes the recipient's User.id at issuance and is displayed as MEGA ID: {id} on the certificate.
- **Instructor** - deliberately *not* required to have a MEGA ID (Instructor.megaIdUserId is optional) - a person can be credited on a course/certificate by name alone before ever creating an account.

What's absent [Future]

- No OAuth/SSO providers - Credentials is the only one configured.
- No email verification at registration.
- No password reset / forgot-password flow.
- No rate limiting on login or registration.

- No account deactivation/deletion flow anywhere in the app.

User Roles

Status legend: **[Implemented]** Implemented - **[Planned]** Designed/approved, not yet implemented - **[Gap]** Known gap/issue - **[Future]** Future/planned Last verified: 2026-08-28, against the current codebase.

All seven roles below are **[Implemented]** implemented and stored identically: a `UserRole` row (`{ userId, role }`, plain string, `@@unique([userId, role])`). A single MEGA ID (User) can hold **multiple roles at once** - see [MEGA_ID.md](#). `dashboard/page.tsx` picks which dashboard to show using a fixed priority order (below), not a strict one-role-per-account rule.

PLATFORM_ADMIN

Gated everywhere by `requirePlatformAdmin()` (`src/lib/authorize.ts`).

- Sees the **Platform Administration Command Center** dashboard - real counts (schools, organizations, teachers, students, courses, certificates, MEGA IDs by role), the schools/organizations verification queues inline, and a "Platform Insights" panel that explicitly lists metrics not yet available rather than showing invented numbers.
- Verifies schools and organizations (`/admin/schools`, `/admin/organizations`, `POST /api/admin/{schools,organizations}/{id}/verify`).
- Can view **any** certificate's preview page, not just their own.
- Seeded fixture: `admin@megaedu.local`.
- **[Gap]** No route or UI exists to grant/revoke `PLATFORM_ADMIN` from within the app - only via `seed.ts` or direct database access.

SCHOOL_ADMIN

- Registers a school and becomes its first `SchoolAdmin` (a school can have more than one admin).
- Full dashboard (`DashboardClient.tsx`, tabbed: Profile / Programs / News / Opportunities / Staff / Students / Finance) - edits school profile, posts programs/news/opportunities, approves pending teachers and students, manages accountants.
- **Phase 2 - Academic Sessions & Grades**, all gated the same way (resolve their own school, no explicit `schoolId` needed since a School Admin only ever administers their own):
 - `/dashboard/setup` - the 5-step Initial School Setup wizard (grades, display names, teacher assignments, student placement, review).
 - `/dashboard/grades` - the grades index, per-grade rosters, and the persistent Pending/Unresolved queue.
 - `/dashboard/sessions/new` - closing the current session and opening a new one, with a preview of exactly what will happen to every student first.
- Gated by `requireSchoolAdmin(schoolId)` on every underlying write route.
- Also has finance access via `requireSchoolFinance` (admin **or** accountant link).

TEACHER

- Registers with school affiliation, or generically then `/api/teacher/join-school`. Starts `approved: false`; a School Admin must approve. Re-joining a school resets approval.
- Dashboard: school status, interests, "Your Students" (Skill management, school-wide - "grade-specific scoping arrives in a later phase" [Future], still true after Phase 2 shipped: Skill creation isn't scoped to a teacher's `TeacherGradeAssignments`), enrolled courses with certificate links.
- Can enroll in and complete MEGA Academy courses.
- Can add Skill records to any approved student at their school.
- **Phase 2:** can be assigned to grades via `TeacherGradeAssignment` (done by a School Admin during Initial Setup), but has no dashboard view of their own assignment or their assigned students' grade rosters - that surface belongs to the School Admin today. [Future]

STUDENT

- Registers with school affiliation, or generically + `/api/student/join-school`. Same approval gate as Teacher.
- Dashboard: school status, interests, read-only Skills list, enrolled courses with certificate links.
- Certificate recipient for course completions.
- **Phase 2:** is placed into `GradeHistory` rows by a School Admin; has no dashboard visibility into their own current grade, promotion history, or session status. [Future]

PARENT

- Registers requiring the child to already exist as a Student (matched by email), or generically + `/api/parent/link-child` to add more children. Can link multiple children.
- Dashboard: read-only view of each linked child's school, grade (legacy `gradeLevel` text, not the Phase 2 structured grade), and approval status; a link-another-child prompt.
- No write actions beyond linking children.
- **Phase 2:** no visibility into a linked child's `GradeHistory` at all. [Future]

ORGANIZATION_ADMIN

- Registers directly, or generically + `/api/organizations/create-for-admin`.
- Dashboard (`OrgDashboard.tsx`, tabbed: Courses / Opportunities / Finance): creates and manages courses, posts opportunities, manages accountants.
- Gated by `requireOrgAdmin(organizationId)`; course-content routes additionally use `requireCourseOwner(courseId)`.
- [Gap] An unverified organization can still create, edit, and - as far as the code enforces - publish a course; nothing checks `Organization.verified` before publishing or allowing enrollment (confirmed via a fresh code search - zero matches). See [KNOWN_GAPS.md](#).

ACCOUNTANT

- **Not self-registerable.** Granted by a School Admin or Organization Admin via email (auto-adds the `ACCOUNTANT` role in a transaction).
- Dashboard: lists every school/organization the person has finance access to, states plainly that no transaction data exists yet since payments aren't built.
- Gated by `requireSchoolFinance/requireOrgFinance` - deliberately check **both** the Admin link and the Accountant link. An Admin retains full authority (finance included); a pure Accountant gets finance access **only**.

School Accountant vs. Organization Accountant [Implemented]

These are two distinct join tables and two distinct grants, not one shared "accountant" concept:

School Accountant	Organization Accountant
<code>MoSchoolAccountant (userId, schoolId)</code>	<code>OrganizationAccountant (userId, organizationId)</code>
<code>GrpPOST /api/schools/[id]/accountants</code> (School by Admin)	<code>POST /api/organizations/[id]/accountants</code> (Org Admin)
<code>AutrequireSchoolFinance(schoolId)</code> helper	<code>requireOrgFinance(organizationId)</code>
Scopes to one school's finance surface	that one organization's finance surface

A person can hold either, both (for different institutions), or neither - the `ACCOUNTANT` `UserRole` is the same for both, but access is always scoped per-institution via the join table, never global.

Dashboard role-priority order [Implemented]

`dashboard/page.tsx` checks roles in this fixed order and renders the first match: `PLATFORM_ADMIN` -> `SCHOOL_ADMIN` -> `TEACHER` -> `STUDENT` -> `PARENT` -> `ORGANIZATION_ADMIN` -> `ACCOUNTANT`, falling back to a generic "not linked to anything yet" message if none match. Explicitly commented in the code as "a simple MVP priority order, not a permission hierarchy" - a person with multiple roles only ever sees one dashboard per visit.

Approval workflows summary

Who	Approved by	Re-triggered by
Teacher	School Admin	Re-joining a school
Student	School Admin	Re-joining a school
School	Platform Admin (verified flag)	Never - no re-verification flow
Organization	Platform Admin (verified flag)	Never
Accountant	Granted directly, no approval queue	N/A

Authentication & Authorization

Status legend: **[Implemented]** Implemented - **[Planned]** Designed/approved, not yet implemented - **[Gap]** Known gap/issue - **[Future]** Future/planned Last verified: 2026-08-28, against the current codebase.

Login / MEGA ID authentication **[Implemented]**

`src/lib/auth.ts` - NextAuth (next-auth v4) with a single `CredentialsProvider` ("MEGA ID"): email + password, `bcrypt.compare` against `User.passwordHash`. `PrismaAdapter` is attached, but session strategy is **JWT**, not database sessions.

- `jwt` callback: on sign-in, copies `id` and `roles` (the array of `UserRole.role` strings) onto the token.
- `session` callback: copies `id` and `roles` from the token onto `session.user`.
- `pages.signIn`: `"/login"` - a custom login page.

Session handling **[Implemented]**

Every server-side check reads the session the same way:

```
const session = await getServerSession(authOptions); const userId = (session?.user as any)?.id; const roles = (session?.user as any)?.roles;
```

The `as any` casts exist because next-auth's default `Session["user"]` type doesn't know about `id/roles`. `src/types/next-auth.d.ts` exists as the ambient-type augmentation point, though the inline casts are still used throughout rather than relying on it everywhere.

Role checks **[Implemented]**

The simplest form - `roles?.includes("SCHOOL_ADMIN")` - is used where there's no specific resource to check against (e.g. gating who's allowed to create a school via `create-for-admin`). For anything scoped to a specific school/organization/course, a `requireX` helper is used instead (below).

School / Organization / Finance / Platform authorization - the `requireX` helper suite **[Implemented]**

`src/lib/authorize.ts` - every write route that needs "does this session have permission to act on this specific resource" calls one of these, rather than inlining the check. Each returns the authorized `userId` or `null` (never throws).

Function	Checks	Used for
----------	--------	----------

<code>requireSchoolAdmin(schoolId)</code>	Session + a <code>SchoolAdmin</code> row for (<code>userId</code> , <code>schoolId</code>)	Every <code>/api/schools/[id]/*</code> write route, including all Phase 2 routes: <code>academic-sessions</code> , <code>academic-sessions/rollover</code> , <code>grades</code> , <code>teacher-assignments</code> , <code>grade-placements</code> , <code>grade-decisions</code> , <code>grade-rollover</code>
<code>requireOrgAdmin(organizationId)</code>	Session + an <code>OrganizationAdmin</code> row	Every <code>/api/organizations/[id]/*</code> write route
<code>requireCourseOwner(courseId)</code>	Resolves the course -> its <code>organizationId</code> -> org-admin check	Course content routes (modules, lessons, publish toggle)
<code>requirePlatformAdmin()</code>	Session + <code>roles.includes("PLATFORM_ADMIN")</code> <code>/api/admin/*</code>	Everything under <code>/admin</code> and <code>/api/admin/*</code>
<code>requireSchoolFinance(schoolId)</code>	Session + (<code>SchoolAdmin</code> row or <code>SchoolAccountant</code> row)	Finance-only routes/tabs for a school
<code>requireOrgFinance(organizationId)</code>	Session + (<code>OrganizationAdmin</code> row or <code>OrganizationAccountant</code> row)	Finance-only routes/tabs for an organization

Deliberate design note (see [PRODUCT_RULES.md](#)): the finance helpers check *both* the Admin and Accountant relationships on purpose - an Admin keeps full authority (finance included), a bare Accountant gets finance access *only*.

Access patterns not covered by `authorize.ts` [Implemented]

Some checks are simple/specific enough to stay inlined rather than factored into a shared helper:

- **Certificate preview access** - `certificate.recipientUserId === userId || roles.includes("PLATFORM_ADMIN")`, inlined in the page.
- **Course enrollment/completion ownership** - `enrollment.teacher?.userId === userId || enrollment.student?.userId === userId`, inlined per route.
- **Promotion roster's closed-session access** (Phase 2) - `/dashboard/grades/[schoolGradeId]?session=<id>` resolves the target session (active by default, or the specific one given) and validates it belongs to the requester's own school via the same School Admin resolution the page already does; the underlying write (`recordGradeDecision()`) doesn't care about session status at all.

Platform administration [Implemented]

`requirePlatformAdmin()` gates `/admin/schools`, `/admin/organizations`, `/api/admin/schools/[id]/verify`, `/api/admin/organizations/[id]/verify`, and the Platform Admin dashboard branch of `dashboard/page.tsx`.

What's absent [Future]

- No rate limiting on login, registration, or any other route.
- No CSRF protection beyond what NextAuth provides for its own endpoints.
- No session revocation mechanism (JWT sessions live until expiry; no server-side "log out everywhere").
- No audit log of authorization *decisions* (as distinct from the data-level `GradeHistoryAudit` trail for grade decisions specifically - see [GRADES_AND_PROMOTION.md](#)).

Product Rules & Architectural Decisions

This document collects every business rule and architectural decision that was **explicitly discussed and approved** during this project's design work - not inferred, not assumed. Each entry states the rule, why it exists, and where it applies. Treat this as the tie-breaker when a future change seems to conflict with existing behavior: if it's here, it was a deliberate choice, not an oversight.

Status legend: **[Implemented]** Implemented - **[Planned]** Designed/approved, not yet implemented - **[Gap]** Known gap/issue - **[Future]** Future/planned Last verified: 2026-08-28, against the current codebase. Read this file before modifying any business logic - see [DEVELOPMENT_GUIDELINES.md](#).

Data integrity & migration discipline

Never use a Prisma `enum` **[Implemented]**

Rule: every status/type/role field is a plain `String`, with valid values documented in a comment above the field. **Why:** SQLite's Prisma connector doesn't support native enums, even unused ones - this broke a migration once ("this bit us once already in Phase 1") and the rule was carried forward explicitly into Phase 2's design brief. **Applies to:** every model in `schema.prisma` - `UserRole.role`, `School.subscriptionTier`, `Certificate.issuerType`, `AcademicSession.status`, `GradeHistory.status`, etc.

Additive-first migrations, verify before cleanup **[Implemented]**

Rule: new schema changes are added without touching existing fields/models first. Before applying anything that could conflict with existing data (like a new `@@unique` constraint), write a one-off script, run it against the real database, confirm there's no conflicting data, *then* apply and delete the script. **Why:** established during the `skill` duplicate-prevention fix (checked for existing duplicate rows before adding `@@unique([studentId, addedByUserId, name])`) and restated as the required approach for the entire Phase 2 schema addition. **Applies to:** all schema changes going forward.

Bulk writes use one transaction, not a loop of individual commits - but the duplicate-skipping pattern inside it is SQLite-specific **[Gap]**

Rule: a bulk write (many rows, each individually validated/skipped on conflict) runs as one `prisma.$transaction(async (tx) => {...})` with a per-item try/catch inside it, not a bare for loop of unwrapped `prisma.create()` calls. Measured directly: on `grade-placements/teacher-assignments`, 200 sequential unwrapped creates took ~15.3s (~77ms/row) vs. ~177ms (~0.9ms/row) wrapped in one transaction - ~86x faster. The same pattern was built transactional *from the start* for `grade-decisions` (100 students, 328ms real HTTP call) and the session rollover sweep (120 students, 365ms real HTTP call). **Why this needs revisiting before Postgres:** the per-item try/catch relies on a caught statement error (e.g. a unique-constraint violation) *not* poisoning the rest of the transaction - verified true on SQLite (a deliberate collision-then-continue test still committed the row that came after the collision). **This is not true on Postgres:** there, any failed statement aborts the whole transaction until an explicit `ROLLBACK/SAVEPOINT`, so every subsequent `tx.create()` call would itself start failing and get miscounted as skipped even when it was actually valid. Since `.env.example/schema.prisma` both mark Postgres as the intended production target, **this pattern must be re-checked (likely reworked to pre-filter duplicates before the transaction) before that migration**. Full list of affected routes in [KNOWN_GAPS.md](#). **Applies to:** `POST /api/schools/[id]/grade-placements`, `POST /api/schools/[id]/teacher-assignments`. **Not applicable to**

`grade-decisions` or the rollover sweep, which validate eligibility *before* opening the transaction and never intentionally hit a duplicate mid-transaction - a genuinely different, Postgres-safe pattern (see below).

Legacy fields are retired by disuse, not by deletion [Implemented]

Rule: `Student.gradeLevel` stays in the schema permanently as a fallback, simply no longer written to once a school completes Phase 2 setup - it is explicitly *not* a field with a breaking migration planned. **Why:** different schools will adopt the structured grade model at different times in a real deployment, so there is no single moment where dropping the field is safe for everyone at once. **Applies to:** `Student.gradeLevel` specifically; the same reasoning should be applied to any future "structured replacement for a free-text field" situation.

Snapshot fields vs. live lookups

Freeze display text at the moment it matters; never live-lookup identity data into a permanent record [Implemented]

Rule: when a record represents a permanent, potentially-shared/printed fact (a certificate, an audit trail entry), any text that names another entity (a person, school, or organization) is copied into a `*Snapshot` field at creation time, not read live through a foreign key relation each time it's displayed. **Why:** a later rename (a school changing its name, a course being retitled) must never silently alter something that's already been issued or already happened. This was the founding design decision behind the `Certificate` model's `recipientNameSnapshot`, `recipientMegaIdSnapshot`, `issuerNameSnapshot`, `associatedSchoolNameSnapshot`, `instructorNameSnapshot` fields, and was explicitly re-applied and confirmed correct when designing `GradeHistoryAudit`'s `previousStatus/previousOutcomeGradeId/newStatus/newOutcomeGradeId` as plain strings rather than live FK relations to `SchoolGrade`. **Applies to:** `Certificate` (all `*Snapshot` fields), `GradeHistoryAudit` (all `previous*/new*` fields).

Logos are the deliberate exception - always live [Implemented]

Rule: unlike name/text fields, a logo is looked up live from the referenced record (`School.logoUrl`), never snapshotted. **Why:** a school/organization updating its logo should apply retroactively to how their name is displayed everywhere, including on previously-issued certificates - this is the opposite instinct from the name-freezing rule above, and was deliberately called out as such. **Applies to:** `CertificateDocument`'s partner-logo rendering (`buildCertificateViewModel` in `certificateView.ts`).

Never hand-type or guess a logo/name; degrade gracefully when the real field is empty [Implemented]

Rule: a school/organization's logo on a certificate must come from that record's actual `logoUrl` field - never invented, never a placeholder image. When it's genuinely absent (as it is for every school/organization today - `Organization` has no `logoUrl` field at all, and no `School.logoUrl` is populated anywhere), show the entity's name in an elegant text treatment instead of a broken image or empty box. **Why:** stated explicitly as a requirement for the certificate template; generalizes to any future feature that displays an optional uploaded asset. **Applies to:** `CertificateDocument.tsx`'s partner-logo block.

The "one audited/gatekept write path" pattern

Every certificate is issued through exactly one function [Implemented]

Rule: `issueCourseCertificate()` (`src/lib/certificates.ts`) is the only code path that may create a `Certificate` row for a course. It guarantees the snapshot fields are always populated at issuance. **Applies to:** currently called only from the enrollment-completion route, atomically alongside marking the enrollment complete (same transaction) - "an enrollment should never end up 'complete' with no certificate, or vice versa."

Every GradeHistory decision is audited, including the first one [Implemented]

Rule: `recordGradeDecision()` (`src/lib/gradeHistory.ts`) is the only code path allowed to change a `GradeHistory` row's status/outcomeGradeId. Every call, in one transaction, updates the row **and** inserts a `GradeHistoryAudit` row capturing the previous and new state - including the very first decision ever recorded on a row, not just later corrections. **Why:** makes it structurally impossible to change a grade decision without leaving a record. Verified repeatedly: an isolated first-decision test (0 audits before, 1 after, correct previous-state capture); a real 100-student bulk Promotion batch through the actual API route producing exactly 100 matching audit rows. **Applies to:** [Implemented] the Student Promotion workflow (`/dashboard/grades/[schoolGradeId]`), `POST /api/schools/[id]/grade-decisions`) is the only feature that calls this. Initial Setup's first-time placements deliberately do **not** go through it - see the next rule.

New placements are creation, not decisions - don't over-audit [Implemented]

Rule: when a student is placed into a grade for the first time (Initial Setup) or carried forward automatically into a new session (rollover), that's a direct `GradeHistory` row creation (status: "ENROLLED", no `decidedAt/outcomeGradeId`) - **not** a call to `recordGradeDecision()`. Only an actual decision changing an *existing* row's outcome (promotion, repeat, transfer, leave) goes through the audited helper. **Why:** this was a self-correction made explicitly during design - the first draft of the plan said Initial Setup should call `recordGradeDecision()`, then was corrected on the reasoning that a brand-new row has no "previous state" to audit against; auditing a creation as if it were a decision would be misleading, not more rigorous. **Applies to:** Initial Setup step 5 (`POST /api/schools/[id]/grade-placements`), the New Session rollover's carry-forward sweep (`carryForwardEligibleStudents()` in `src/lib/gradeRollover.ts`), and the Pending/Unresolved queue's "manually place" action (same endpoint, reused). Verified in all three cases: `decidedAt: null` and zero `GradeHistoryAudit` rows on the resulting placements.

The carry-forward sweep is idempotent and re-runnable, not a one-shot [Implemented]

Rule: `carryForwardEligibleStudents()` can be safely called more than once against the same target session - re-running it after nothing has changed places zero additional students and throws no error, because it always checks "does this student already have a row in the target session" before creating one. **Why:** resolving a Pending/Unresolved student can happen at any point after a new session already exists (record their missing decision today, next week, whenever) - the sweep needs to be re-runnable on demand (a "Place eligible students now" button), not just fired once automatically at rollover time. **Applies to:** `POST /api/schools/[id]/grade-rollover` (on-demand re-run) and the automatic call inside `POST /api/schools/[id]/academic-sessions/rollover`. Verified directly: first run placed 10/10 eligible students; an immediate second run with no state changes placed 0, threw no error, and a duplicate check confirmed zero students with more than one row in the target session.

Never guess when confidence is low

Legacy grade-text matching returns null, not a best-effort guess [Implemented]

Rule: the utility that matches free-text grade values ("Grade 6", "Class 6", "VI", "Nursery") to a `GradeReference` code must return `null` - not a low-confidence guess - whenever it can't confidently match. Unmatched students go to a manual-assignment queue, never a silently wrong auto-match. **Why:** stated as an explicit requirement; wrong auto-placement into a grade is worse than requiring a human to resolve it. **Applies to:** `matchLegacyGradeText()` (`src/lib/gradeMatching.ts`), used during Initial School Setup step 5. Verified with 20+ real inputs, including confirming the full Roman numeral range (I-X, including subtractive notation IV/IX) resolves correctly, and that genuinely ambiguous input ("KG" alone, "Room 6B", out-of-range numbers) correctly returns `null`.

Students without a recorded decision are never silently defaulted [Implemented]

Rule: when starting a new `AcademicSession`, any student whose most recent `GradeHistory` row is still `ENROLLED` (no decision ever recorded - not `outcomeGradeId` set) must be excluded from automatic placement into the new session entirely, and instead listed in a clearly visible, **persistent** Pending/Unresolved queue. The School Admin must explicitly resolve each one before that student gets a grade in the new session. **Why:** the same "never guess" principle applied to session rollover - an unresolved student silently carried forward (or silently left out) would hide a real gap in the school's record-keeping. **Applies to:** `findPendingStudents()` (`src/lib/gradeRollover.ts`), surfaced on `/dashboard/grades`. Deliberately session-agnostic: it always looks at a student's *true most recent* `GradeHistory` row, however many sessions back that is - verified across a real 3-session chain where a student pending after session A->B remained correctly pending (referencing session A, not the empty intervening session B) after a second rollover B->C.

A pending student can be resolved two different ways, both legitimate [Implemented]

Rule: (1) "Record the missing decision" - go back to the student's old, now-closed session roster and apply a real Promote/Repeat/Transfer/Leave decision (audited via `recordGradeDecision()`), then a follow-up sweep places them in the new session. (2) "Manually place them" - place them directly into the current session without ever deciding the old row; the old row stays permanently `ENROLLED` with zero audit rows, an honest gap rather than a fabricated decision. **Why:** both were explicitly specified as valid resolutions; forcing only one path would either lose historical accuracy (if manual placement were disallowed) or block legitimate fast-path corrections (if only the audited path were allowed). **Applies to:** the Pending/Unresolved queue's two actions on `/dashboard/grades`. Verified with both paths exercised on different students in the same test run, confirmed distinguishable at the database level by audit-row count (1 vs. 0).

Access control

Every write route is gated by a `requirex` helper that returns the `userId` or null [Implemented]

Rule: no route inlines its own ad-hoc permission check against `session.user.roles` for admin-style access - it calls a shared `requireSchoolAdmin` / `requireOrgAdmin` / `requireCourseOwner` / `requirePlatformAdmin` / `requireSchoolFinance` / `requireOrgFinance` from `src/lib/authorize.ts`. **Why:** keeps the permission logic in one place per relationship type, testable and auditable independently of any one route. See [AUTHENTICATION_AND_AUTHORIZATION.md](#). **Applies to:** every `/api/schools/[id]/*`, `/api/organizations/[id]/*`, `/api/admin/*`, and course-content write route - including every Phase 2 route added since.

Finance access is additive to Admin, never a substitute check for it [Implemented]

Rule: `requireSchoolFinance/requireOrgFinance` deliberately check **both** the Admin link and the Accountant link - an Admin retains full authority (finance included), while a pure Accountant (not also an Admin) gets finance access **only**. Nothing outside finance-scoped routes should ever call these functions to gate a non-finance action. **Applies to:** the Finance tab in `OrgDashboard/DashboardClient`, `AccountantDashboard`.

A certificate's designed preview is recipient-or-platform-admin only [Implemented]

Rule: `/dashboard/certificates/[id]/preview` checks `certificate.recipientUserId === userId` **or** `PLATFORM_ADMIN` role - never viewable by an arbitrary logged-in user who guesses an id. This was verified live with three different sessions (recipient, platform admin, unrelated teacher - the last one correctly redirected away). **Applies to:** the certificate preview route only.

Certificate system: two verification surfaces, kept deliberately separate

`/verify/[code]` and the certificate preview serve different audiences and must both keep working [Implemented]

Rule: `/verify/[code]` is the public, no-login-required page for a third party (an employer, another school) to confirm a certificate is real - plain text, unstyled, permanent. The designed `CertificateDocument` preview at `/dashboard/certificates/[id]/preview` is for the certificate owner viewing their own achievement. Neither replaces the other; dashboards link "View certificate" to the owner-facing preview, while the plain page remains reachable by its code-based URL for sharing. **Why:** explicitly stated as two different purposes serving two different audiences, not a redundant older/newer pair. **Applies to:** `src/app/verify/[code]/page.tsx` (must never be modified casually) and `src/components/certificate/CertificateDocument.tsx`.

Skill duplicate-prevention: same person + same skill + same student = duplicate; different people = independent attestation [Implemented]

Rule: `@@unique([studentId, addedByUserId, name])` on `Skill` - a teacher double-clicking "Add Skill" can't create a duplicate row, but two different teachers independently crediting the same student with the same skill is meaningful and must both be recorded. **Why:** multiple attestations from different people carry real information (corroboration); the same person repeating an action is just an accident to be prevented. **Applies to:** `Skill` model and its creation route, which catches the resulting P2002 constraint violation and returns `{ ok: true, alreadyExists: true }` rather than a raw error - the client treats this identically to a successful add.

Session-aware UI, without duplicating auth logic

The homepage Register button is session-aware, computed server-side, no client auth re-check [Implemented]

Rule: the homepage hero's Register button becomes a non-interactive, visually dimmed element (native `title` tooltip, no `href`, `aria-disabled`) when the visitor is already logged in, computed from `getSession` in the same server component that renders the rest of the homepage - not a second client-side session check. **Why:** keeps

a single source of truth for "is this visitor logged in" per page render, avoids a flash-of-wrong-state on load. **Applies to:** `src/app/page.tsx`.

Reporting incomplete data honestly

Never show a placeholder statistic - omit it or explicitly label it "coming soon" [Implemented]

Rule: the Platform Admin dashboard's "Platform Insights" panel explicitly lists metrics that can't yet be computed (revenue/payments, growth trends, moderation actions) with the reason why, rather than fabricating a number or hiding the gap silently. **Why:** stated as a hard requirement when building the dashboard - "use only data and actions that already exist in the system... where a metric cannot yet be calculated from existing data, leave it out or clearly mark it as a future dashboard item." **Applies to:** `PlatformAdminDashboard.tsx`, and by extension, this documentation's own status-tagging convention.

Course access & commerce - what's actually decided vs. not

Free enrollment is implemented; nothing about pricing structure has been decided yet [Future]

Rule: `Course.priceCents` exists and defaults to 0; enrollment for a free course works end-to-end. Enrollment for a course with `priceCents > 0` is **explicitly blocked** with an error message - no payment flow exists to complete it. **What is *not* an approved rule, despite sounding like it should exist by now:** there is no "Free vs. Premium course" tier system, no "course bundle by grade" concept, and no defined "enrollment access method" beyond the single free-enrollment path described above. A direct search of the codebase for any bundle/premium/tier concept turned up nothing, and no prior design discussion in this project's history approved one either. If a future request asks to "preserve" these as existing rules, that request is working from an incorrect premise - check here first rather than assuming they were previously decided. **Applies to:** `POST /api/courses/[courseId]/enroll`. See [COURSES_AND_ENROLLMENTS.md](#) and [KNOWN_GAPS.md](#).

Academic Sessions

Status legend: **[Implemented]** Implemented - **[Planned]** Designed/approved, not yet implemented - **[Gap]** Known gap/issue - **[Future]** Future/planned Last verified: 2026-08-28, against the current codebase. Part of "Phase 2: Academic Sessions & Grades" - see [GRADES_AND_PROMOTION.md](#) for the promotion/rollover mechanics, and [PRODUCT_RULES.md](#) for the underlying design principles.

What it is **[Implemented]**

An `AcademicSession` represents one school year (or term) for a single school - the container everything else in Phase 2 is scoped to: which teachers are assigned to which grades, and which grade every student is placed in, are both per-session.

Schema **[Implemented]**

`AcademicSession` - `id`, `schoolId` (FK), `name`, `startDate`, `endDate`, `status` (default "ACTIVE"). Valid status: ACTIVE | CLOSED.

The one-ACTIVE-session-per-school rule **[Implemented]**

A school may have at most one `ACTIVE` session at a time. This is enforced **at the application level, not a database constraint** (SQLite can't express a partial unique index) - `POST /api/schools/[id]/academic-sessions` checks for an existing `ACTIVE` session before creating a new one, and if one exists it returns that session with `alreadyActive: true` (HTTP 200, not an error) rather than creating a second one. Verified live with a genuine two-tab race condition: both tabs submitted the create form before either refreshed, and only one session was ever created - the second request correctly folded into a no-op.

Creating the first session **[Implemented]**

Part of [Initial School Setup](#) - a School Admin creates the school's first `AcademicSession` as a prerequisite step before configuring grades. `POST /api/schools/[id]/academic-sessions`.

Closing a session and opening the next - "rollover" **[Implemented]**

`POST /api/schools/[id]/academic-sessions/rollover` (`/dashboard/sessions/new` in the UI) closes the current `ACTIVE` session and opens a new one, in one transaction:

1. The prior session's `status` is set to `CLOSED`.
2. A new session is created with `status: "ACTIVE"`.
3. Every student whose most recent decision was `COMPLETED` or `REPEATED` **with a real outcome grade recorded** is automatically placed into the new session at that grade (a direct `GradeHistory` creation, not a decision - see [PRODUCT_RULES.md](#)).

4. Anyone still **ENROLLED** with no decision ever recorded is **left unplaced** and surfaces in the persistent Pending/Unresolved queue - see [GRADES_AND_PROMOTION.md](#).

Verified: a full end-to-end run with 6 students in a mixed state (2 promoted, 1 repeated, 1 left, 2 undecided) produced exactly the right split at the database level, and a real 120-student timing run through the actual HTTP route completed in 365ms.

Grade decisions happen independently of session status [Implemented]

The Student Promotion workflow (per-grade rosters, see [GRADES_AND_PROMOTION.md](#)) has **no dependency on session status at all**, and no dependency on other grades being finished first. A School Admin can promote/repeat/transfer/leave students in Grade 6 today and not touch Grade 7 until next week - each grade's roster is handled independently, at whatever pace the school actually works at. The only thing that ever forces a session boundary is the School Admin explicitly choosing to start a new session via the rollover flow.

One added wrinkle, also implemented and tested: the Promotion roster page can target a **closed** session too (`/dashboard/grades/[schoolGradeId]?session=<id>`), reached only from the Pending/Unresolved queue's "Record decision ->" link - this is how a School Admin resolves a student whose decision was never made before the session closed, without reopening the session itself.

What's built vs. not

Piece	Status
<code>AcademicSession</code> model	[Implemented]
One-ACTIVE-per-school rule (app-enforced)	[Implemented] verified with a real race condition
First-session creation (Initial Setup)	[Implemented]
Rollover (close prior, open new, auto-carry-forward)	[Implemented] verified at 6-student and 120-student scale
Pending/Unresolved queue, persistent across any number of rollovers	[Implemented] verified across a 3-session chain
Any UI to rename/edit/delete a past session	[Future] not built - sessions are create-and-close only
Per-session reporting/analytics (e.g. how many students per grade over time)	[Future] not built

Explicitly out of scope for Phase 2

Per the original design brief: certificates, MEGA Academy courses, Opportunities, and Notifications were untouched by this phase. A future grade-completion certificate (`issueGradeCertificate()`, see [CERTIFICATES.md](#)) would eventually read from `GradeHistory/AcademicSession`, but that integration doesn't exist yet.

Grades & Promotion

Status legend: **[Implemented]** Implemented - **[Planned]** Designed/approved, not yet implemented - **[Gap]** Known gap/issue - **[Future]** Future/planned Last verified: 2026-08-28, against the current codebase. See [ACADEMIC_SESSIONS.md](#) for the session side, and [PRODUCT_RULES.md](#) for the underlying design principles (snapshot fields, never-guess matching, audited decisions).

Why this exists **[Implemented]**

`Student.gradeLevel` is a free-text field - schools historically typed "Grade 6", "Class 6", "VI", whatever they liked, with no structure and no history. Phase 2 replaces this with a structured, auditable model, while leaving `gradeLevel` in place permanently as a legacy fallback (see [PRODUCT_RULES.md](#)) - schools adopt the new structure on their own timeline, and nothing forces a breaking migration.

Schema **[Implemented]**

Six models total (`AcademicSession` has its own doc). Full field lists in [DATABASE.md](#).

- **GradeReference** - the platform-wide, fixed grade ladder. Seeded once: PP1, PP2, PP3, Y1, Y2, ..., Y10 (13 rows, order 1-13). Not school-editable.
- **SchoolGrade** - a school's opt-in to one `GradeReference`, with its own `displayName` (e.g. "Class 6" for Y6).
- **TeacherGradeAssignment** - per-session teacher-to-grade link.
- **GradeHistory** - a student's grade placement for one session; the permanent record. Unique on (`studentId`, `academicSessionId`).
- **GradeHistoryAudit** - append-only log of every decision ever written to a `GradeHistory` row.

The audited write-path: `recordGradeDecision()` **[Implemented]**

`src/lib/gradeHistory.ts` - the **only** code path anywhere in the app allowed to change a `GradeHistory` row's status/outcomeGradeId. In one transaction: reads the current state, writes the new state, inserts a `GradeHistoryAudit` row capturing both - including the very first decision ever made on a row, not just later corrections. Rejects an invalid status before writing anything.

Verified: a targeted test isolated exactly the "first decision" case - 0 audit rows before, 1 after, `previousStatus`: "ENROLLED", `previousOutcomeGradeId`: `null` captured correctly.

The legacy matching utility: `matchLegacyGradeText()` **[Implemented]**

`src/lib/gradeMatching.ts` - matches free text ("Grade 6", "Class 6", "VI", "Nursery", "UKG") to a `GradeReference` code. Handles the full Roman numeral range I-X including subtractive notation (IV->4, IX->9 - verified explicitly, not assumed), ordinal suffixes ("6th Grade"), and pre-primary keywords (nursery->PP1, lkg->PP2, ukg->PP3). **Returns null - never a guess - for anything ambiguous or out of range** ("Grade 11", "KG" alone, "Room 6B", spelled-out numbers): verified with over 20 real inputs including deliberately tricky ones.

Initial School Setup - 5-step guided flow [Implemented]

/dashboard/setup, gated by resolving the School Admin's own school (same access pattern as every other school-admin surface). Verified end-to-end live, with a real mix of matchable and unmatchable student records.

1. **Session** - create the school's first `AcademicSession` if none exists yet.
2. **Configure grades** - pick which `GradeReferences` the school uses, creating `SchoolGrade` rows. `POST /api/schools/[id]/grades`.
3. **Display names** - set each grade's label, defaulting to the code, editable. Same endpoint as step 2.
4. **Assign teachers** - bulk-pick teacher -> grade(s) for the current session. `POST /api/schools/[id]/teacher-assignments`.
5. **Assign students** - runs `matchLegacyGradeText()` against every approved student's `gradeLevel`. Confident matches are pre-filled for one-click bulk confirmation; everything else goes into a manual queue with multi-select + bulk-assign. `POST /api/schools/[id]/grade-placements` - **a direct `GradeHistory` creation, status: "ENROLLED", not routed through `recordGradeDecision()`**, since a first-time placement isn't a decision changing an existing row (see [PRODUCT_RULES.md](#) for why this distinction matters).
6. **Review & confirm** - live counts (grades configured, teachers assigned, students placed, students still unmapped).

Verified with a real 5-student scenario: matching split 3 confident / 2 manual exactly as designed; database-level check confirmed all 5 resulting rows had `decidedAt: null` and produced **zero** `GradeHistoryAudit` rows, proving the direct-creation path was actually used.

Student Promotion workflow - per grade, per session [Implemented]

/dashboard/grades/[schoolGradeId] - a School Admin opens one `SchoolGrade`'s roster for the active session, sees every currently-ENROLLED student, multi-selects some, and applies one decision to the whole batch:

Decision	<code>GradeHistory.status</code>	<code>outcomeGradeId</code>	Meaning
Promote	COMPLETED	required - the target grade	Student finished this grade and moves up
Repeat	REPEATED	required - same grade as today	Student stays in this grade next session
Transfer	TRANSFERRED	none	Student leaves this school for another
Leave	LEFT	none	Student leaves this school entirely

Example: Y6 -> Y7 - a School Admin opens the Grade 6 roster, selects a group of students, clicks Promote. The default suggested target grade is the nearest later `GradeReference` the school has actually configured (Y7, by order) - shown pre-selected in an editable dropdown, so the admin can override it (e.g. skip a grade, or promote into a grade out of strict sequence) if that's genuinely the right call.

Example: Y6 -> Repeat Y6 - same roster, Repeat instead. The dropdown defaults to the *current* grade rather than the next one.

Example: Y6 -> Transfer / Y6 -> Leave - no grade dropdown appears at all; these decisions record why the student is no longer part of this school's grade progression, with no destination grade.

Bulk selection: every decision applies to the *entire currently-selected set* in one action - there's a "select all" toggle, and each student can be checked/unchecked individually. **Every row in the batch goes through recordGradeDecision() inside one prisma.\$transaction** (built transactional from the start for this step, per an explicit requirement) - verified with a real 100-student batch: 328ms total, decided: 100, skipped: 0, and a database check confirmed exactly 100 matching GradeHistoryAudit rows.

Individual outcomes within one submission: not supported - one decision type and one outcome grade apply to the whole selected batch per submission. To give different students different outcomes, submit separate batches (e.g. select the promoted group, apply Promote; then select the repeaters, apply Repeat).

Eligibility is re-checked server-side, not just trusted from the UI: submitting a gradeHistoryId that's already been decided (or doesn't belong to the school) is silently excluded and counted in skipped, never double-decided. Verified with a mixed batch (one already-decided id + one genuinely eligible id): {decided: 1, skipped: 1}, and the already-decided row's audit count stayed at exactly 1.

The final-grade edge case: promoting a student already in the school's highest configured grade shows no default target (empty dropdown, not a bad guess), with an explanatory note ("No later grade is configured..."), and blocks submission with a clear error until the admin picks a grade manually or chooses a different decision. Verified live.

Undecided students & next-session activation [Implemented]

A student who is never decided stays ENROLLED in their current session's GradeHistory row indefinitely - nothing forces a decision. When the School Admin starts a new session (rollover), that student is **excluded from automatic placement** and appears in the persistent **Pending/Unresolved** queue on /dashboard/grades, which stays visible for as long as they remain unresolved - not just at the moment of rollover.

Two ways to resolve a pending student, both implemented: 1. **Record the missing decision** - the queue links directly to that student's grade roster, scoped to the closed session (?session=<id>), where the admin applies a real decision. This alone doesn't place them in the new session - a follow-up "Place eligible students now" action (or the next rollover) picks them up. 2. **Manually place them** - select the student directly in the queue, pick a grade, and place them into the current session immediately, skipping the old decision entirely. Their old row stays permanently ENROLLED with zero audit rows - a deliberate, honest gap in the record (the school chose not to record why), not an error.

Verified across a 3-session chain (a specific edge case worth calling out): a student left pending after session A->B remained correctly pending through B->C as well - even though they never had any row in session B at all - and findPendingStudents() still correctly traced back to their true last real placement in session A. Resolving them afterward placed them directly into session C, correctly skipping the empty session B.

Repeated grades [Implemented]

A repeat isn't an edit to the existing row - it's the REPEATED decision on the current session's row (with outcomeGradeId pointing at the *same* grade), and a **new** GradeHistory row gets created in the next session at that grade via the same carry-forward mechanism as a promotion. Nothing is overwritten; the history of "repeated Grade 6 in 2026, then completed it in 2027" is fully preserved across two separate rows.

Audit records [Implemented]

Every decision - promote, repeat, transfer, or leave - produces exactly one `GradeHistoryAudit` row, capturing `previousStatus/previousOutcomeGradeId` -> `newStatus/newOutcomeGradeId`, who (`changedByUserId`) and when (`changedAt`). This table is genuinely append-only: no route anywhere updates or deletes a `GradeHistoryAudit` row. New placements (Initial Setup, rollover carry-forward, manual pending placement) are **not** audited, by design - see [PRODUCT_RULES.md](#) for the reasoning.

Deliberately out of scope

Per the original Phase 2 brief: certificates, MEGA Academy courses, Opportunities, and Notifications were not touched by this work.

Certificates

Status legend: **[Implemented]** Implemented - **[Planned]** Designed/approved, not yet implemented - **[Gap]** Known gap/issue - **[Future]** Future/planned Last verified: 2026-08-28, against the current codebase.

Model **[Implemented]**

Certificate (see [DATABASE.md](#) for the full field list) deliberately keeps **recipient, instructor, and issuer as separate concepts** - never conflated:

- `recipientUserId` + `recipientNameSnapshot`/`recipientMegaIdSnapshot` - who earned it.
- `instructorId?` + `instructorNameSnapshot?` - who taught it, if recorded. Instructor doesn't require a MEGA ID.
- `issuerType` (MEGA_EDU | ORGANIZATION | SCHOOL | JOINT) + `issuerOrganizationId?`/`issuerSchoolId?` + `issuerNameSnapshot` - who issued it.
- `associatedSchoolId?` + `associatedSchoolNameSnapshot?` - a separate "affiliated school" context (e.g. the recipient's own school), distinct from the issuer, shown as its own detail line rather than folded into the issuer.

All *Snapshot fields are frozen at issuance time and never re-derived from live data - see [PRODUCT_RULES.md](#). The one deliberate exception is the partner logo, looked up live via `issuerSchool.logoUrl/associatedSchool.logoUrl` (no snapshot field exists for it).

`gradeHistoryId?` is reserved for grade certificates but has **no live Prisma relation declared** - a bare, unlinked `String?` column. Even though `GradeHistory` now fully exists and is heavily used (Phase 2), this field remains untouched - grade-certificate issuance was deliberately kept out of Phase 2's scope.

Issuance - course certificates **[Implemented]**

`issueCourseCertificate()` (`src/lib/certificates.ts`) is the only code path that creates a `Certificate` for a course. Called from `POST /api/enrollments/[enrollmentId]/complete`, inside the same transaction that marks the `CourseEnrollment` complete. Sets `issuerType`: "ORGANIZATION" unconditionally (the only issuer type reachable today). A `CERTIFICATE_ISSUED` notification follows (best-effort, outside the transaction).

Grade certificates **[Planned]** (designed, not implemented)

`issueGradeCertificate()` doesn't exist. The `certificates.ts` doc comment explicitly earmarks this as future work, parallel to the course path, once `GradeHistory` exists - which it now does, but the issuance function itself was never built, by explicit scope decision (Phase 2's brief excluded certificates entirely).

Display: `CertificateDocument` **[Implemented]**

`src/components/certificate/CertificateDocument.tsx` - a true-to-size A4 landscape (1123x794px on-screen, @page { size: 297mm 210mm } for print) layout, driven by a `CertificateViewModel` from `buildCertificateViewModel()` (`src/lib/certificateView.ts`), reading only snapshot fields plus the live logo

lookup.

Layout, top to bottom: MEGA.EDU wordmark (left, or centered alone with no partner) / partner institution logo-or-name (right) -> eyebrow ("Certificate of Completion" or "Certificate of Achievement" depending on `isGradeCertificate`) -> "This certificate is proudly presented to" -> recipient name (verified to wrap gracefully up to ~100 characters without overflow) -> MEGA ID: `{id}` -> "for successfully completing" -> course/grade title -> detail row (Completion Date, Instructor if present, Affiliated School if present and distinct from the partner) -> signature lines -> Certificate ID + "Verify this certificate through MEGA.EDU" + a marked-but-empty QR code space.

Course vs. grade wording [Planned] - the template branches on `!!certificate.gradeHistoryId`, but since no grade certificate has ever been issued (no issuance function exists), this branch is exercised only by design review, never by real data.

Partner logo resolution [Implemented] - for `issuerType SCHOOL/Joint`, the partner is `issuerSchool` (real `logoUrl` if present, else name-only); for `ORGANIZATION/Joint` fallback, always name-only, since `Organization` has no `logoUrl` field in the schema at all ([Gap] see [KNOWN_GAPS.md](#)). For MEGA_EDU, no partner - the hero re-centers around the MEGA.EDU wordmark alone.

Where it's shown [Implemented]

- `/dashboard/certificates/[id]/preview` - the designed, owner-facing view. Access-gated to the certificate's recipient or a Platform Admin, verified live with three different sessions. Wrapped in `CertificateScaler.tsx`, which scales the fixed-size document to fit the viewport.
- `/verify/[code]` - public, no-login, plain-text verification page. Deliberately separate and unstyled; see [PRODUCT_RULES.md](#).
- Linked from `TeacherDashboard.tsx/StudentDashboard.tsx`'s course list ("View certificate ->" -> the preview page, via `certificate.id`) and from the course "learn" page's completion banner (-> `/verify/[code]`).

Verification [Implemented] / [Future]

The Certificate ID (`verificationCode`) is shown prominently on both surfaces. A space for a QR code is marked but empty on the certificate document - no QR generation library has been added.

PDF generation [Future]

Not built. The certificate design was explicitly staged to be approved as an in-browser preview first, with PDF export deferred to a later pass.

School/organization logos on certificates [Implemented] (mechanism) / [Gap] (data)

The mechanism is fully built and correct (see Partner logo resolution above), but **no school or organization in the current database actually has a logo** - every `School.logoUrl` is null, and `Organization` has no such field at all. Every certificate rendered today shows the name-only fallback, which was explicitly designed for and verified to look correct, not a broken/unfinished state.

Courses & Enrollments (MEGA Academy)

Status legend: **[Implemented]** Implemented - **[Planned]** Designed/approved, not yet implemented - **[Gap]** Known gap/issue - **[Future]** Future/planned Last verified: 2026-08-28, against the current codebase.

Model [Implemented]

Course (belongs to an `Organization`, optionally an `EducationalApproach` and an `Instructor`) -> `CourseModule` -> `Lesson`, plus `CourseEnrollment` (either `teacherId` or `studentId`, never neither) and its optional 1:1 `Certificate`. Full field list in [DATABASE.md](#).

Course creation [Implemented]

An `Organization Admin` creates a course from their dashboard (`OrgDashboard.tsx` -> `POST /api/organizations/[id]/courses`, gated by `requireOrgAdmin`): title, description, and an optional plain-text instructor name (creates an `Instructor` row inline, no MEGA ID required). New courses are free by default (`priceCents: 0`) and unpublished (`published: false`).

Course publishing [Implemented] / [Gap]

From `/dashboard/courses/[courseId]/manage` (gated by `requireCourseOwner`), the admin adds `CourseModules` and `Lessons`, then toggles published. **Publishing requires at least one lesson** - the toggle is disabled otherwise. **[Gap]** An unverified organization can still publish - no route checks `Organization.verified` before allowing this (confirmed via a fresh code search). See [KNOWN_GAPS.md](#).

Organizations vs. Schools in MEGA Academy [Implemented]

Every course belongs to an `Organization`, never a `School` - schools have no course-authoring capability in the current system. `Certificate.associatedSchoolId` links a certificate to the recipient's school as *informational context*, not as course ownership.

Free courses [Implemented]

Enrollment for a free course (`priceCents: 0`) works end-to-end: `POST /api/courses/[courseId]/enroll` -> idempotent (`alreadyEnrolled: true` on re-submission) -> learn page -> `POST /api/enrollments/[enrollmentId]/complete` -> certificate issued atomically.

Premium courses, school bundles, grade-specific bundles, personal purchase [Future] (not implemented - see note below)

None of these exist in the codebase. A direct search for "premium" or "bundle" anywhere in the source returns zero matches. What actually exists:

- `Course.priceCents` - a plain integer field. If it's `> 0`, enrollment is **explicitly blocked** with the error "Paid course enrollment isn't available yet. This course is not free." No purchase flow of any kind runs.
- No concept of a "bundle," "tier," or "grade-specific" course package exists in the schema or any route.
- No prior design discussion in this project approved a bundle/premium model either - this isn't a regression, it's simply undesigned territory. See [PRODUCT_RULES.md](#) for the explicit note on this.

If a school-wide or grade-specific bundle purchase model is wanted, it would need to be designed from scratch - there's no partial implementation to build on top of.

Enrollment [Implemented]

`POST /api/courses/[courseId]/enroll`: - Requires login and a `Teacher` or `Student` profile (403 otherwise - School Admins, Parents, Org Admins, Accountants can't enroll). - Requires the course to be published. - Blocks any priced course (see above). - Idempotent.

This is the **only** enrollment access method that exists - there is no invite-only enrollment, no school-assigned bulk enrollment, and no "grade-gated" course visibility.

Course completion & certificates [Implemented]

`/courses/[slug]/learn` (login required) shows every module/lesson and a `CompleteButton` once content exists. `POST /api/enrollments/[enrollmentId]/complete`: - Ownership-checked, idempotent (`alreadyCompleted: true` returns the existing certificate). - In one transaction: `progress: 100 + completedAt`, then `issueCourseCertificate()` - an enrollment can never end up "complete" with no certificate or vice versa. - `CERTIFICATE_ISSUED` notification follows (best-effort).

The learn page shows a completion banner linking to `/verify/[verificationCode]`; the dashboards link to the designed preview instead (see [CERTIFICATES.md](#)).

What's designed but not wired [Future]

- **Paid enrollment / payment integration.** `Subscription/Payment` are modeled, no processor connected.
- **Progress tracking beyond 0/100.** `CourseEnrollment.progress` exists but nothing updates it incrementally.
- **Course reviews/ratings, richer search/filtering.** Not present.

Explicitly out of scope for Phase 2

Per the Phase 2 design brief, this system was untouched by the Academic Sessions & Grades work - no changes were made here, and grade-certificate issuance (once built) would be a parallel path alongside `issueCourseCertificate()`, not a replacement.

API Reference

Status legend: **[Implemented]** Implemented - **[Planned]** Designed/approved, not yet implemented - **[Gap]** Known gap/issue - **[Future]** Future/planned Last verified: 2026-08-28, against the current codebase - every route below exists in `src/app/api/**/*.route.ts` as documented. This is a complete inventory; nothing here is invented.

All routes are **[Implemented]** implemented. "Auth" means the caller must be logged in (`getServerSession`). "Authz" is the specific `requireX` helper (see [AUTHENTICATION_AND_AUTHORIZATION.md](#)) or inline check used, if any beyond plain login. Response bodies are JSON; a successful response generally includes `{ ok: true, ... }`, an error `{ error: string }`.

Authentication & Registration

Method	Endpoint	Purpose	Auth	Authz	Notes
*	<code>/api/auth/[...nextAuth]</code>	NextAuth handler (sign in/out, session, CSRF)	-	-	Standard NextAuth internals
POST	<code>/api/auth/register</code>	Generic single-role registration	-	-	<code>{name, email, password, role}</code> , role is one of STUDENT, TEACHER, PARENT, SCHOOL_ADMIN, ORGANIZATION_ADMIN. 409 if email exists. Teacher/Student/Parent get an unaffiliated profile created immediately
POST	<code>/api/auth/registerTeacher</code>	Teacher registration + school affiliation in one step	-	-	Requires <code>schoolId</code> of an already-verified school; 400 otherwise
POST	<code>/api/auth/registerStudent</code>	Student registration + school affiliation	-	-	Same verified-school requirement

POST	/api/auth/register-parent	Parent registration + link to an existing child	-	-	Requires <code>childEmail</code> to already belong to a <code>Student</code> ; 400 if not found
POST	/api/auth/register-organization	Organization Admin registration + org creation in one step	-	-	Slugifies <code>orgName</code> , appends a random suffix on collision

Post-registration affiliation

Method	Endpoint	Purpose	Auth	Authz	Notes
POST	/api/teacher/join-school	Affiliate a teacher with a school	[Implemented]	inline (own <code>Teacher</code> row)	Resets <code>approved: false</code>
POST	/api/student/join-school	Affiliate a student	[Implemented]	inline	Resets <code>approved: false</code>
POST	/api/parent/link-child	Link additional child by email	[Implemented]	inline (own <code>Parent</code> row)	Idempotent - <code>alreadyLinked: true</code>
POST	/api/schools/create-for-admin	Create school for an already-registered <code>SCHOOL_ADMIN</code> role holder with no school yet	[Implemented]	inline (<code>roles.includes</code>)	409 if already administers one
POST	/api/organizations/create-for-admin	Same, for <code>ORGANIZATION_ADMIN</code>	[Implemented]	inline	409 if already administers one
POST	/api/schools/register-school+admin	Alternate school+admin creation path	-	-	Same shape as <code>register-organization</code> for schools

Platform Admin

Method	Endpoint	Purpose	Auth	Authz	Notes
POST	/api/admin/schools/verify	Verify school	[Implemented]	requirePlatformAdmin	Set <code>verified: true</code> , notifies school admins
POST	/api/admin/organizations/[id]/verify	Verify organization	[Implemented]	requirePlatformAdmin	Set <code>verified: true</code> , notifies org admins

Schools - directory & profile

Method	Endpoint	Purpose	Auth	Authz	Notes
GET	/api/schools/search	Public school search (typeahead)	-	-	Filters <code>verified: true</code> , <code>isActive: true</code> ; max 20 results
PATCH	/api/schools/[id]	Update school profile fields	[Implemented]	requireSchoolAdmin	<code>description</code> , <code>contactEmail</code> , <code>contactPhone</code> , <code>location</code> , <code>gradesOffered</code>
POST	/api/schools/[id]/add-program	Add program	[Implemented]	requireSchoolAdmin	{ <code>name</code> , <code>description</code> }
POST	/api/schools/[id]/post-news-item	Post news item	[Implemented]	requireSchoolAdmin	Fires <code>notifySchoolCommunity()</code> (best-effort)
POST	/api/schools/[id]/post-opportunities	Post opportunities	[Implemented]	requireSchoolAdmin	{ <code>title</code> , <code>description</code> , <code>type</code> , <code>deadline?</code> , <code>applyUrl?</code> }
GET/POST	/api/schools/[id]/link-accountants	Link accountants School Accountant access	[Implemented]	requireSchoolAdmin	POST by email; auto-adds <code>ACCOUNTANT</code> role if missing; 404 if no MEGA ID with that email, <code>alreadyGranted: true</code> if already linked

Schools - staff & students

Method	Endpoint	Purpose	Auth	Authz	Notes
GET	/api/schools/[id]/staff	List approved students + skills	[Implemented]	inline (approved teacher or admin at this school)	Not scoped by grade - any approved teacher sees the whole roster
POST	/api/schools/[id]/teachers/[teacherId]/approve	Approve pending teacher	[Implemented]	requireSchoolAdmin	Sends STAFF_APPROVED notification
POST	/api/schools/[id]/students/[studentId]/approve	Approve pending student	[Implemented]	requireSchoolAdmin	Sends STUDENT_APPROVED notification
POST	/api/schools/[id]/students/[studentId]/skills	Create student with a skill	[Implemented]	inline (approved teacher or admin at the student's own school)	Catches P2002 -> {ok: true, alreadyExists: true} instead of an error

Schools - Academic Sessions & Grades (Phase 2)

All gated by `requireSchoolAdmin(id)`. See [ACADEMIC_SESSIONS.md](#) and [GRADES_AND_PROMOTION.md](#) for full behavioral detail - this table is structural.

Method	Endpoint	Purpose	Important request data	Important response / errors
POST	/api/schools/[id]/academicSessions	Create (or update) session	{name, startDate, endDate}	alreadyActive: true (HTTP 200, no-op) if an ACTIVE session already exists - not an error
POST	/api/schools/[id]/academicSessions/roll	Close current session, open a new one, sweep eligible students forward	{name, startDate, endDate}	400 if no ACTIVE session exists to close; response includes placed: <count>
POST	/api/schools/[id]/grades	Rules, insert SchoolGrade selection/display names	{grades: [{gradeReferenceId, displayName}]}	Additive-only, never deletes an existing SchoolGrade

POST	/api/schools/[id]/teacher-assignments	Backfill assignments for one session	{academicSessionId, assignments: [{teacherId, schoolGradeId}]}	One transaction; invalid/foreign ids and duplicates silently counted in skipped, not errored
DELETE	/api/schools/[id]/teacher-assignments/[assignmentId]	Remove assignment		404 if not found or wrong school
POST	/api/schools/[id]/grade-placements	Full re-placements first-time GradeHistory rows	{academicSessionId, placements: [{studentId, schoolGradeId}]}	Direct creation, not recordGradeDecision(); one transaction; duplicates counted in skipped
POST	/api/schools/[id]/grade-decisions	Full apply Promotion decision	{gradeHistoryIds: string[], status: "COMPLETED, REPEATED, TRANSFERRED, or LEFT", outcomeSchoolGradeId}	Every row routed through recordGradeDecision() inside one transaction; ineligible ids (wrong school, already decided) pre-filtered and counted in skipped; 400 if zero ids are eligible
POST	/api/schools/[id]/grade-history	Order and run of the carry-forward sweep against the current session	-	400 if no ACTIVE session; idempotent - re-running with nothing new to place returns placed: 0, never an error

Organizations

Method	Endpoint	Purpose	Auth	Authz	Notes
POST	/api/organizations/[id]/courses	Create course	[Implemented]	requireOrgAdmin	Optional inline Instructor creation by name
POST	/api/organizations/[id]/opportunities	Create opportunity	[Implemented]	requireOrgAdmin	Same shape as the school version

GET/POST	/api/organizations/[id]/accounts	Organization Accountant access	[Implemented]	requireOrgAdmin	Same pattern as the school version
----------	----------------------------------	--------------------------------	---------------	-----------------	------------------------------------

Courses & Enrollment

Method	Endpoint	Purpose	Auth	Authz	Notes
PATCH	/api/courses/[courseId]	Update course fields (including publish toggle)	[Implemented]	requireCourseOwn	Accepts any of title, description, published, priceCents
POST	/api/courses/[courseId]/modules	Add module	[Implemented]	requireCourseOwn	order auto-set to current module count
POST	/api/courses/[courseId]/modules/[moduleId]/lessons	Add lesson	[Implemented]	requireCourseOwn	404 if module doesn't belong to the course
POST	/api/courses/[courseId]/enroll	Enroll caller (as their Teacher or Student profile)	[Implemented]	inline (must have a Teacher or Student profile)	404 if course not published; 400 if priceCents > 0 ([Gap] paid enrollment not implemented); idempotent - alreadyEnrolled: true
POST	/api/enrollments/[enrollmentId]/complete	Mark enrollment complete and issue a certificate	[Implemented]	inline (must own the enrollment)	One transaction: progress: 100 + issueCourseCertificate() idempotent - alreadyCompleted: true with the existing certificate

Identity layer

Method	Endpoint	Purpose	Auth	Authz	Notes
--------	----------	---------	------	-------	-------

POST	/api/interests	Add a self-declared interest	[Implemented]	inline (own User)	Idempotent - alreadyExists: true
DELETE	/api/interests/[id]	Remove an interest	[Implemented]	inline (must own it)	404 if not owned

Notifications

Method	Endpoint	Purpose	Auth	Authz	Notes
POST	/api/notifications	Mark all of read caller's notifications read	[Implemented]	inline	Bulk updateMany
GET	/api/notifications/user/badge count	User badge count		-	Returns {count: 0} for a logged-out caller rather than erroring

Not implemented / not applicable

No routes exist for: deleting a User/School/Organization/Course, deactivating a school/organization (isActive is read but never set by any route), payment processing, PDF certificate export, QR code generation, or grade-certificate issuance. See [KNOWN_GAPS.md](#).

Testing

Status legend: **[Implemented]** Implemented - **[Planned]** Designed/approved, not yet implemented - **[Gap]** Known gap/issue - **[Future]** Future/planned Last verified: 2026-08-28, against the current codebase.

Current state: no automated test suite [Gap]

There is no test framework installed - no Jest, Vitest, Playwright, or Cypress in `package.json`, no config file for any of them, and no `*.test.*` or `*.spec.*` files anywhere in the repository (re-confirmed for this documentation pass). `npm run lint` (Next.js/ESLint) and the TypeScript compiler are the only automated checks that exist.

What verification practice is actually used instead [Implemented]

1. **Typecheck as a gate.** `npx tsc --noEmit -p tsconfig.json` after every non-trivial change.
2. **Live verification against the running dev server, using seeded demo accounts** (see below), including negative cases - e.g. confirming an unrelated user is correctly redirected away from something they shouldn't see, not just that the right person can see it.
3. **Throwaway verification scripts, run once and deleted.** For anything touching real data - checking for existing duplicate rows before a new constraint, timing a bulk route, proving a sweep is idempotent - a one-off script under `prisma/`, run with `npx tsx`, output inspected, then deleted. Nothing like this is left behind in the repository.

Development database testing [Implemented]

All verification runs against the real `prisma/dev.db` SQLite file with real Prisma calls - not a mocked/in-memory database. This is deliberate: it's how the SQLite-vs-Postgres transaction-abort issue (see [PRODUCT_RULES.md](#)) was actually discovered, not theorized.

Seeded demo accounts (the de facto manual test fixtures) [Implemented]

From `prisma/seed.ts` (idempotent - safe to re-run):

Role	Email	Password	Notes
Platform Admin	<code>admin@megaedu.local</code>	ChangeMe123! (or <code>SEED_ADMIN_*</code> env vars)	
School Admin	<code>demo.school@megaedu.local</code>	DemoSchool123!	Administers "Sunrise Academy" (verified)

Teacher	demo.teacher@megaedu.local	DemoTeacher123!	Pre-approved at Sunrise Academy
Student	demo.student@megaedu.local	DemoStudent123!	Pre-approved gradeLevel "Grade 9"
Parent	demo.parent@megaedu.local	DemoParent123!	Linked to the demo student
Organization Admin	demo.org@megaedu.local	DemoOrg123!	Administers "MEGA Academy Labs" (verified), one published course

Plus a verified demo school, a published demo course, two demo opportunities, five educational approaches, and 13 seeded `GradeReference` rows.

Phase 2 verification performed [Implemented]

Every one of the six Phase 2 steps was independently verified with real evidence, not just typechecked, before being considered done:

Step	Verification performed
Schema	<code>prisma validate</code> clean; explicit checkpoint review before proceeding
<code>recordGradeDecision()</code>	Isolated first-decision test (0->1 audit rows, correct previous-state capture); invalid-status rejection confirmed pre-write
<code>matchLegacyGradeText()</code>	20+ real inputs including the full Roman numeral range I-X with subtractive notation (IV, IX), messy whitespace, and deliberately ambiguous/out-of-range cases confirmed to correctly return <code>null</code>
Initial School Setup	Full live run with 5 real students (varied <code>gradeLevel</code> text); confident/manual split verified exactly; database check confirmed direct-creation (no audit rows)

Student Promotion	All four decisions tested live; 100-student bulk batch through the real API route (328ms, 100/100 audited); mixed already-decided/eligible batch correctly reported <code>{decided: 1, skipped: 1}</code> ; final-grade edge case (no default target) verified
New Session rollover	Full 6-student mixed-outcome scenario verified at the database level; both Pending/Unresolved resolution paths proven distinct; idempotency of the carry-forward sweep proven directly (second run: 0 placed, no error, zero duplicates); 120-student real HTTP timing run (365ms); a 3-session chain proving pending-tracking survives an intervening session with zero rows for that student

Each test used real throwaway fixture data against the actual database and, where relevant, the actual HTTP routes (not just direct function calls) - and every fixture was fully cleaned up afterward, with a final row-count check confirming zero residue.

Known testing limitations [Gap]

- No automated regression protection - every verification above was manual and one-time; a future change could silently break any of it without a test suite catching it.
- No load/concurrency testing beyond the single deliberate two-tab race condition test (the `alreadyActive` session-creation check).
- No testing against PostgreSQL - everything above ran on SQLite; the one known behavioral difference (transaction-abort-on-error) is documented but not empirically verified against a real Postgres instance, since none exists in this project.
- No accessibility, performance, or cross-browser testing.

Future/planned [Future]

No specific automated testing strategy has been decided or approved. If taken up, natural starting points given the codebase's own architecture: unit tests for the `src/lib` "sole write-path" functions (`issueCourseCertificate`, `recordGradeDecision`, `matchLegacyGradeText`, `carryForwardEligibleStudents`) - already informally proven correct by the manual tests above; integration tests for the `authorize.ts` helpers; end-to-end tests for the manually-verified flows.

Deployment

Status legend: **[Implemented]** Implemented - **[Planned]** Designed/approved, not yet implemented - **[Gap]** Known gap/issue - **[Future]** Future/planned Last verified: 2026-08-28, against the current codebase. This document describes only what actually exists - no production infrastructure has been set up, and nothing below should be read as implying otherwise.

Local development **[Implemented]**

```
npm install npm run dev # next dev - starts on http://localhost:3000
```

Other `package.json` scripts: `npm run build` (Next.js production build), `npm run start` (serve a production build), `npm run lint` (ESLint), `npm run db:push` (`prisma db push` - applies schema changes directly, no migration files), `npm run db:seed` (`tsx prisma/seed.ts` - idempotent demo data), `npm run db:studio` (Prisma Studio, a local DB browser).

Database **[Implemented]** (dev) / **[Future]** (production)

Development uses SQLite, a single file (`prisma/dev.db`), configured via `DATABASE_URL="file:./dev.db"`. `schema.prisma`'s own header comment and `.env.example` both mark this as dev-only:

"Local dev default (SQLite file). For production, point this at a Postgres connection string (e.g. from Neon: <https://neon.tech>) and change `provider = "sqlite"` to `provider = "postgresql"` in `prisma/schema.prisma`."

[Gap] This switch has never been made or tested. See the SQLite/Postgres transaction-behavior note in [KNOWN_GAPS.md](#) and [PRODUCT_RULES.md](#) - some bulk-write routes rely on a SQLite-specific behavior (a caught statement error doesn't poison the rest of a transaction) that does **not** hold on Postgres and would need rework before that switch.

Environment variables **[Implemented]**

From `.env.example` - the complete, real list; nothing else is read anywhere in the app:

Variable	Purpose	Dev default
<code>DATABASE_URL</code>	Prisma connection string	<code>file:./dev.db</code>
<code>NEXTAUTH_SECRET</code>	JWT signing secret for NextAuth	placeholder, must be replaced for any real deployment (<code>openssl rand -base64 32</code>)
<code>NEXTAUTH_URL</code>	Canonical app URL NextAuth uses for callbacks	<code>http://localhost:3000</code>

SEED_ADMIN_EMAIL	Platform Admin account created by db:seed	admin@megaedu.local
SEED_ADMIN_PASSWORD	Same	ChangeMe123! - must be changed before seeding a real environment

Production deployment [Future]

Nothing has been deployed. No hosting platform, no CI/CD pipeline, no Dockerfile, no `next.config.js` production overrides beyond Next.js defaults exist in this repository. This is genuinely unstarted work, not an oversight in documentation.

Known deployment requirements (inferred from the codebase, not yet acted on) [Future]

Before any real deployment, based on what the code actually requires:

1. A Postgres database (per the schema comment above), with `schema.prisma's provider` switched and `npx prisma db push` (or a proper migration) run against it.
2. A real `NEXTAUTH_SECRET` and `NEXTAUTH_URL` matching the deployed domain.
3. A real `SEED_ADMIN_EMAIL/SEED_ADMIN_PASSWORD` before running the seed script, or skip seeding demo/fixture data entirely in production.
4. Review of the bulk-write transaction pattern noted above before relying on it under Postgres.
5. Whatever the hosting platform requires for a standard Next.js 14 App Router app (Node.js runtime; no edge-specific code is used anywhere in this codebase, so no special edge-runtime configuration is needed).

PostgreSQL considerations [Gap]

Specifically flagging what's known to need attention, not a general "be careful" note:

- The `grade-placements` and `teacher-assignments` bulk-write routes catch a unique-constraint violation (P2002) *inside* an open transaction and continue looping. This works on SQLite (verified directly) but would misbehave on Postgres, where a failed statement aborts the whole transaction until rollback. See [PRODUCT_RULES.md](#) for the full explanation and the routes affected.
- No Prisma enums are used anywhere specifically because SQLite doesn't support them - this constraint disappears on Postgres, but there's no plan to introduce enums retroactively; the `plain-String` convention is intended to stay regardless of database.

Known Gaps & Issues

Last verified: 2026-08-28 - every item below was actively re-checked against the current codebase before being listed (grep/read, not assumption). If an item is ever fixed, move it out of this file rather than leaving it marked open.

Data model gaps

`Organization` has no logo field at all [Gap]

Unlike `School` (which has `logoUrl`, just unpopulated), `Organization` has no such column in the schema. The certificate system already handles this gracefully (name-only fallback), so it isn't breaking anything today, but it's a real structural gap if organization logos are ever wanted. See [DATABASE.md](#), [CERTIFICATES.md](#).

`School.isActive` / `Organization.isActive` are read but never written [Gap]

Both fields default to `true` and are used as a filter in two places (`schools/search`, Platform Admin dashboard counts), but **no route anywhere ever sets either to false**. There is no deactivation action in the app. Confirmed via a direct search: `isActive` appears in exactly three files, all reads.

Enforcement gaps

Organization verification isn't enforced on course publishing or enrollment [Gap]

`Organization.verified` exists and is set by a Platform Admin, and the Org Admin dashboard tells the admin their courses won't "go live" until verified - but nothing in `PATCH /api/courses/[courseId]` (the publish toggle) or `POST /api/courses/[courseId]/enroll` actually checks `verified`. Confirmed with a fresh search across both routes: zero references to `verified`. An unverified organization can publish and receive enrollments today. See [USER_ROLES.md](#), [COURSES_AND_ENROLLMENTS.md](#).

Database portability

SQLite-specific transaction behavior in two bulk-write routes [Gap]

`POST /api/schools/[id]/grade-placements` and `POST /api/schools/[id]/teacher-assignments` catch a unique-constraint violation inside an open `prisma.$transaction` and continue the loop - this relies on SQLite tolerating a caught statement error without poisoning the rest of the transaction, which is **not** true on Postgres (the documented production target). Measured, not assumed: verified directly that SQLite continues correctly; reasoned (not yet tested against a real Postgres instance, since none exists in this project) that Postgres would abort the transaction after the first collision. Needs rework before any Postgres migration. Full detail: [PRODUCT_RULES.md](#), [DEPLOYMENT.md](#). **Not affected:** `grade-decisions` and the rollover carry-forward sweep - both validate eligibility *before* opening the transaction and never intentionally hit a duplicate mid-transaction, a genuinely different and Postgres-safe pattern.

Testing

No automated test suite exists [Gap]

No Jest/Vitest/Playwright/Cypress, no config, no `*.test.*` files anywhere in the repository - confirmed by checking `package.json` and searching the tree. See [TESTING.md](#) for what verification practice is used instead (typecheck gate, live browser runs against seeded fixtures, throwaway `tsx` scripts).

Commerce / payments

No payment processor is integrated [Future]

`Subscription/Payment` are modeled but unconnected. Paid course enrollment (`priceCents > 0`) is explicitly blocked rather than attempted. No "premium," "bundle," or "grade-specific package" concept exists anywhere in the code - confirmed via a direct search returning zero matches. See [PRODUCT_RULES.md](#), [COURSES_AND_ENROLLMENTS.md](#).

Certificates

PDF export and QR code generation are not built [Future]

The certificate visual design is finished and approved as an in-browser preview; downloading a PDF and generating a scannable QR code were both explicitly deferred and never started. A space for the QR code is marked on the certificate layout but rendered empty. See [CERTIFICATES.md](#).

Grade-certificate issuance doesn't exist [Future]

`Certificate.gradeHistoryId` is a reserved, unlinked column. No `issueGradeCertificate()` function exists, even though `GradeHistory` (its intended data source) is now fully built. This was a deliberate Phase 2 scope exclusion, not an oversight.

Phase 2 (Academic Sessions & Grades) - role visibility gaps

Teachers, Students, and Parents have no dashboard visibility into Phase 2 data [Future]

All of Initial Setup, Promotion, and New Session rollover are School-Admin-only surfaces. A Teacher can't see their own `TeacherGradeAssignments` or the roster of a grade they're assigned to; a Student can't see their own `GradeHistory/current grade`; a Parent can't see a linked child's structured grade at all (the Parent dashboard still shows only the legacy `gradeLevel` text). None of this was in Phase 2's scope, but it's worth tracking as the natural next surface.

`skill` isn't scoped to a teacher's grade assignment [Future]

Any approved teacher at a school can add a `Skill` to any approved student at that school - there's no check against `TeacherGradeAssignment` to restrict this to students in a grade the teacher actually teaches. Pre-dates Phase 2 and wasn't addressed by it.

Authentication

Several standard auth features are absent [Future]

No OAuth/SSO, no email verification at registration, no password reset flow, no login/registration rate limiting, no session revocation ("log out everywhere"), no account deactivation or deletion route for any model. See [AUTHENTICATION_AND_AUTHORIZATION.md](#).

`PLATFORM_ADMIN` can only be granted via the seed script or direct database access [Future]

No in-app route or UI exists to promote a user to Platform Admin, or to revoke it.

Development Guidelines

Last verified: 2026-08-28. This document is for anyone - human or AI-assisted - making changes to this codebase. It's a set of rules earned from how this project has actually been built across Phase 1 and Phase 2, not generic advice.

Before touching business logic

Read [PRODUCT_RULES.md](#) first. Every rule in that file was explicitly discussed and approved, with its rationale preserved. If a change seems to conflict with something documented there, that's a signal to stop and check - not to route around it. Do not change an approved rule unless you've verified the actual implementation proves the rule was never really followed, or a human has explicitly approved changing it.

Before touching architecture

Read [ARCHITECTURE.md](#) first. This codebase has established, consistent conventions (server components fetch Prisma directly; the "one audited/gatekept write path" pattern for anything transactional; `router.refresh()` as the only revalidation strategy). Introducing a different pattern in one corner of the app - a new state manager, a different mutation flow, a bypass of an existing `lib` helper - makes the codebase harder to reason about even if the new code works. Match what's already there unless there's a real reason not to, and if there is, document it.

Scope discipline

- **Do not modify unrelated functionality.** A bug fix doesn't need surrounding cleanup; a one-shot operation doesn't need a helper someone might use someday. This project has consistently been built in small, explicitly-scoped steps (see the Phase 2 six-step breakdown in [GRADES_AND_PROMOTION.md/ACADEMIC_SESSIONS.md](#)) - follow that pattern rather than bundling unrelated changes into one pass.
- **Don't add functionality beyond what's asked.** If a request describes 5 steps, build 5 steps - not a 6th "while I'm here" feature. If you notice something that seems missing, flag it rather than silently building it.

Data integrity

- **Preserve historical records.** `GradeHistory`, `GradeHistoryAudit`, and `Certificate` rows are permanent by design - no route deletes them, and none should be added that does, without an explicit, separate decision to do so. If a student leaves a school, that's a new `status` on their record (`TRANSFERRED/LEFT`), never a deleted row.
- **Avoid destructive migrations.** This project's discipline (see [PRODUCT_RULES.md](#)) is additive-first: add new fields/models without touching existing ones, verify no data conflicts with a throwaway script before applying anything with a new constraint, and only clean up afterward - sometimes not even then (`Student.gradeLevel` is intentionally permanent).
- **Test migrations before removing old fields.** Before adding a `@@unique` constraint or any other constraint that could reject existing data, write a one-off script that checks the real database for conflicts first. This exact

practice caught and prevented a real issue during the `Skill` duplicate-prevention fix.

- **Never use a Prisma enum.** SQLite's connector doesn't support them, even unused ones - this has broken a migration before. Use a plain `String` field with the valid values documented in a comment above it.

Don't invent

- **Don't invent data, business rules, or functionality that doesn't exist in the code.** If a request assumes something exists that a search of the codebase doesn't confirm, say so - don't build a plausible-sounding version of it to satisfy the request's framing. (A concrete example from this project's own history: a documentation request assumed "premium courses" and "course bundles" were previously-approved rules; a direct search found zero evidence they were ever built or designed, so the documentation says exactly that instead of inventing a design for them.)
- **Don't invent APIs, database fields, or test results.** Every route, field, and model referenced in `/docs` should be verifiable by reading the actual source - exact names, exact shapes.
- **Don't fabricate historical development information.** This project's git history is a single squashed commit with no granular record - [CHANGELOG.md](#) says so explicitly rather than presenting a reconstructed timeline as authoritative fact.

Verification

- This project has no automated test suite (see [TESTING.md](#)) - the substitute is real verification: `typecheck` (`npx tsc --noEmit`), then exercise the actual change against the running dev server and real database, using throwaway fixture data that gets cleaned up afterward. Don't claim something works because it typechecks; claim it works because you ran it.
- For anything touching a bulk operation or a transaction, verify the transactional behavior directly (does a duplicate get skipped without crashing? does the whole batch commit as one unit?) rather than assuming the code does what it looks like it does.

Documentation

- **Update documentation when an important architectural or business-rule decision changes** - not as an afterthought, and not by leaving the old doc silently stale. If Phase 3 changes how Promotion works, `GRADES_AND_PROMOTION.md` needs to change in the same pass, the way `ACADEMIC_SESSIONS.md` and `GRADES_AND_PROMOTION.md` were fully rewritten (not just patched) once Phase 2's steps went from designed to actually built and verified.
- Distinguish clearly, always: **[Implemented] Implemented** (exists and works, verified against the running code) vs. **[Planned] Designed/approved, not yet implemented** vs. **[Gap] Known gap/issue** vs. **[Future] Future/planned**. Don't blur these to make a feature sound more finished than it is.
- Keep documentation maintainable, not exhaustively verbose - a table beats five paragraphs when the information is the same either way.

Changelog

All notable changes to MEGA.EDU are recorded here, in [Keep a Changelog](#) style (Added / Changed / Fixed), newest first.

A note on how this file was built: the git history for this project is a single squashed Initial commit - there is no granular commit-by-commit history to generate this changelog from mechanically. What follows is a hand-written reconstruction of the real, verified milestones, grouped by feature area. Where a date is given below, it's inferred from the development session's own context (not a git commit timestamp) and should be read as approximate, not authoritative. **Going forward, add a dated entry here for every notable change** - that's the only way this file stays trustworthy once real commit history exists to lean on instead.

Unreleased

Added - Documentation regeneration (2026-08-28, approximate)

- Full /docs refresh establishing an accurate baseline for Phase 1 + Phase 2 as they actually exist in code - not a patch, a verified rewrite of every file against a fresh reading of the codebase.
- `ACADEMIC_SESSIONS.md` and `GRADES_AND_PROMOTION.md` fully rewritten - both previously described Initial Setup/Promotion/New Session rollover as not-yet-built; all three are now complete, so both docs needed a real rewrite, not a status-tag edit.
- `DATABASE.md` fully rewritten with a consistent per-model structure (Purpose / Key fields / Relationships / Constraints / Delete behavior / Currently used), and Phase 2 models' status corrected from "schema-only" to actively used.
- `PRODUCT_RULES.md` updated: several rules that were tagged "designed, not yet implemented" are now marked implemented with their verification evidence; new rules added for the carry-forward sweep's idempotency and the two pending-resolution paths; an explicit note added distinguishing "free course enrollment" (real) from "premium courses / bundles" (never implemented, never previously approved as a rule - a direct codebase search confirms this).
- Four new files: `API.md` (full inventory of all current API routes), `DEPLOYMENT.md` (local dev setup and what's known/unknown about production), `KNOWN_GAPS.md` (consolidated, individually re-verified list of open issues), `DEVELOPMENT_GUIDELINES.md` (rules for future AI-assisted work on this codebase).
- Status legend standardized across all docs to four tiers: [Implemented] Implemented / [Planned] Designed, not implemented / [Gap] Known gap / [Future] Future.

Added - Phase 2: Academic Sessions & Grades - complete (2026-08-28, approximate)

All six steps of the original Phase 2 design brief, each independently verified with real evidence (live browser runs, database-level checks, and timing measurements against the actual API routes) - not just typechecked:

1. **Schema** - six new models (`AcademicSession`, `GradeReference`, `SchoolGrade`, `TeacherGradeAssignment`, `GradeHistory`, `GradeHistoryAudit`), additive, `Student.gradeLevel` untouched. 13 `GradeReference` rows seeded (PP1-PP3, Y1-Y10).
2. **`recordGradeDecision()`** (`src/lib/gradeHistory.ts`) - sole audited write-path for `GradeHistory` decisions.

3. `matchLegacyGradeText()` (`src/lib/gradeMatching.ts`) - free-text-to-grade matching, never guesses, returns `null` on ambiguity.
4. **Initial School Setup** (`/dashboard/setup`) - 5-step wizard: session creation, grade configuration, display names, teacher assignment, student placement (confident-match + manual queue), review.
5. **Student Promotion** (`/dashboard/grades/[schoolGradeId]`) - per-grade roster, bulk Promote/Repeat/Transfer/Leave, every decision audited, transactional bulk writes.
6. **New Session rollover** (`/dashboard/sessions/new`, `/dashboard/grades`) - closes the prior session, opens a new one, auto-carries-forward students with a recorded outcome, and maintains a persistent Pending/Unresolved queue with two distinct resolution paths.

Fixed - Bulk-write performance and UI correctness during Phase 2 hardening (2026-08-28, approximate)

- Wrapped the grade-placements and teacher-assignments bulk-write loops in a single `prisma.$transaction`, cutting a 200-row batch from ~15.3s to ~177ms (~86x). Documented the SQLite-vs-Postgres transaction-abort caveat this pattern relies on.
- Fixed the `SetupWizard`'s session-creation flow to actually surface `alreadyActive: true` to the School Admin (previously silently discarded their input and advanced anyway) - now shows a dismissible notice naming the existing session, verified live with a genuine two-tab race condition.
- Fixed a grammar bug in the Promotion roster's success messages ("Repeatd", "Leaved" from naive string concatenation) - replaced with a proper per-decision message function.

Added - Certificate redesign

- A true-to-size A4 landscape `CertificateDocument` component: MEGA.EDU wordmark + partner school/organization logo-or-name, full recipient/course/issuer hierarchy, course-vs-grade wording support, conditional instructor line/signature, reserved (unbuilt) QR space.
- `buildCertificateViewModel()` (`src/lib/certificateView.ts`) - snapshot-only view model builder, with live logo lookup as the deliberate exception.
- New route `/dashboard/certificates/[id]/preview`, access-gated to the certificate's recipient or a Platform Admin (verified with three different live sessions).
- "View certificate" links on `TeacherDashboard/StudentDashboard` repointed from the plain `/verify/[code]` page to the new designed preview, using the certificate's id; the public `/verify/[code]` page was left completely unchanged - both surfaces now serve their distinct audiences.
- Widened the `certificate` prop types on both dashboards to include `id` (the underlying Prisma query already fetched it via `certificate: true`; only the TypeScript type was too narrow).

Fixed - Skill duplicate prevention

- Added `@@unique([studentId, addedByUserId, name])` to `Skill`, closing a gap where double-clicking "Add Skill" could create duplicate rows. Verified zero existing duplicate rows before applying the constraint.
- The skill-creation route now catches the resulting Prisma `P2002` violation and returns `{ ok: true, alreadyExists: true }` instead of a raw error; the client already treated this as a silent success with no code changes needed.
- Different teachers independently crediting the same student with the same skill remains fully supported (the constraint is scoped per-adder, not per-skill-name alone).

Added - Platform Admin dashboard

- New `PlatformAdminDashboard.tsx`, replacing the generic "your MEGA ID isn't linked to anything" fallback that Platform Admins previously saw on `/dashboard`.
- Real, live-queried counts only: schools/organizations (total, verified, active, pending), teachers/students (total, approved), courses (total, published), certificates issued, MEGA IDs by role - no invented or placeholder statistics.
- Inline pending-verification queues for schools and organizations, reusing the existing `VerifyButton/VerifyOrgButton` components and `/api/admin/*` routes rather than duplicating verification logic.
- A "Platform Insights" panel (originally "Coming soon") that explicitly names metrics not yet computable (revenue/payments, growth trends, moderation actions) instead of hiding or faking them.
- `DashboardHero` gained an optional `title` override so this dashboard could show a static heading instead of the time-of-day greeting, without changing any other role's dashboard.

Changed - Homepage

- "Explore mega.edu" hero button shortened to "Explore"; recolored to orange for visual distinction from the navy "Register" button.
- The "Register" button is now session-aware: for a logged-in visitor it renders as a non-interactive, visually dimmed element (native tooltip: "You already have a MEGA ID") in the exact same layout position, computed server-side from the same request that renders the rest of the homepage. Logged-out visitors see the original active button, unchanged. "Explore" remains active for everyone. Verified in both states with real login/logout sessions.

Added - Documentation (initial pass)

- Initial `/docs` structure: this file plus `PROJECT_OVERVIEW.md`, `ARCHITECTURE.md`, `DATABASE.md`, `USER_ROLES.md`, `MEGA_ID.md`, `PRODUCT_RULES.md`, `CERTIFICATES.md`, `ACADEMIC_SESSIONS.md`, `GRADES_AND_PROMOTION.md`, `COURSES_AND_ENROLLMENTS.md`, `AUTHENTICATION_AND_AUTHORIZATION.md`, `TESTING.md`. (Superseded by the full regeneration entry above once Phase 2 was completed.)

Baseline (pre-existing at the start of this changelog's coverage)

Everything below existed before the entries above and predates any granular history this changelog can reconstruct - grouped by area, not dated:

- **MEGA ID & roles** - unified `User/UserRole` identity, NextAuth credentials+JWT login, the full two-stage registration system (generic + role-specific routes) and post-registration affiliation routes (join-school, link-child, create-for-admin).
- **Schools & Organizations** - directories, profiles, admin/accountant management, Platform Admin verification queues.
- **MEGA Academy** - course authoring (modules/lessons/publish), free-course enrollment, completion tracking, certificate issuance on completion (the original `Certificate` model and `issueCourseCertificate()`).
- **Content** - Programs, News, Opportunities, Resources, Events.
- **Notifications** - the `notify()/notifySchoolCommunity()` system and unread-count bell.

- **Identity layer** - Interests and Skills (pre-dating the duplicate-prevention fix above).
- **Original certificate model** - per `schema.prisma`'s own header comment, an earlier, simpler certificate model existed before being replaced by the current recipient/instructor/issuer-split design (with a backfill script, `prisma/backfill-certificates.ts`, still present in the repo for that migration).